

**SOLVING AXION DYNAMICS USING THE PHYSICS-INFORMED NEURAL
NETWORKS (PINNS)**

MR. WEERACHAI SEEONSAENG

**A THESIS FOR THE DEGREE OF MASTER OF SCIENCE
KHON KAEN UNIVERSITY**

2025

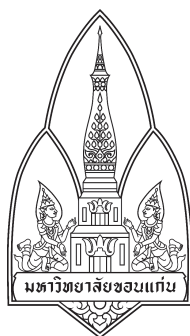
**SOLVING AXION DYNAMICS USING THE PHYSICS-INFORMED NEURAL
NETWORKS (PINNS)**

MR. WEERACHAI SEEONSAENG

**A THESIS SUBMITTED IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE DEGREE OF MASTER OF SCIENCE
IN PHYSICS**

GRADUATE SCHOOL KHON KAEN UNIVERSITY

2025



THESIS APPROVAL
KHON KAEN UNIVERSITY
FOR
MASTER OF SCIENCE
IN PHYSICS

Thesis Title: Solving Axion Dynamics using the Physics-Informed
Neural Networks (PINNs)

Author: Mr. Weerachai Seeonsaeng

Thesis Examination Committee

Assoc. Prof. Dr. Piyabut Burikham	Chairperson
Assoc. Prof. Dr. Chakrit Pongkitivanichkul	Member
Assoc. Prof. Dr. Thanayut Kaewmaraya	Member
Assoc. Prof. Dr. Daris Samart	Member

Thesis Advisor:

..... Advisor
(Assoc. Prof. Dr. Chakrit Pongkitivanichkul)

.....	
(Prof. Dr. Kiatichai Faksri)	(Asst. Prof. Dr. Angkana Boonyued)
Dean, Graduate School	Dean, Faculty of Science

Copyright of Khon Kaen University

วีระชัย สืออ่อนแสง. 2568. การแก้ปัญหาคณิตศาสตร์ของเอกซิมอนโดยใช้เครือข่าย

ประสาทเทียมที่ได้รับข้อมูลจากฟิสิกส์. วิทยานิพนธ์ปริญญาวิทยาศาสตร

มหาบัณฑิต สาขาวิชาฟิสิกส์ บัณฑิตวิทยาลัย มหาวิทยาลัยขอนแก่น.

อาจารย์ที่ปรึกษาวิทยานิพนธ์: รองศาสตราจารย์ ดร. ชคริต พงษ์กิตติคุณกุล

บทคัดย่อ

โครงข่ายประสาทเทียมที่อาศัยข้อมูลฟิสิกส์ (PINNs) เป็นโครงข่ายประสาทเทียมประเภทหนึ่งที่ถูกออกแบบมาเพื่อแก้ปัญหาคณิตศาสตร์ที่ควบคุมด้วยกฎทางฟิสิกส์ แทนที่จะอาศัยข้อมูลเพียงอย่างเดียว โครงข่ายประสาทเทียมที่อาศัยข้อมูลฟิสิกส์จะรวมสมการฟิสิกส์ เช่น สมการเชิงอนุพันธ์ ไว้ในกระบวนการฝึกโดยตรง วิธีการนี้ช่วยให้แบบจำลองสามารถค้นหาคำตอบที่มีทั้งความถูกต้องเชิงตัวเลขและสอดคล้องฟิสิกส์ ในงานวิจัยนี้ เราประยุกต์ใช้โครงข่ายประสาทเทียมที่อาศัยข้อมูลฟิสิกส์กับจักรวาลวิทยาเอกซิมอน เพื่อเพิ่มความแม่นยำและเสถียรภาพของการฝึก เราใช้การฝังคุณลักษณะฟูรีเยร์เพื่อบันทึกพฤติกรรมการแกว่งและบล็อกตักค้ำเพื่อลดปัญหาความชันที่หายไป ผลการศึกษาของเราบ่งชี้ว่าโครงข่ายประสาทเทียมที่อาศัยข้อมูลฟิสิกส์สามารถจำลองวิวัฒนาการของสนามสเกลาร์ของเอกซิมอนได้อย่างแม่นยำ ซึ่งแสดงให้เห็นถึงความสอดคล้องที่ดีเยี่ยมกับคำตอบเชิงตัวเลขแบบดั้งเดิม ผลการวิจัยเหล่านี้เน้นย้ำถึงศักยภาพของโครงข่ายประสาทเทียมที่อาศัยข้อมูลฟิสิกส์ในฐานะทางเลือกที่น่าสนใจสำหรับการแก้สมการเชิงอนุพันธ์ที่ซับซ้อน ยิ่งไปกว่านั้น วิธีการนี้ยังแสดงให้เห็นถึงศักยภาพสำหรับการพัฒนาในอนาคตและการขยายผลไปยังระบบหลายสนาม หรือปัญหาพลวัตไม่เชิงเส้นอื่นๆ ในฟิสิกส์เชิงทฤษฎีสมัยใหม่

Weerachai Seeonsaeng. 2025. **Solving Axion Dynamics using the Physics-Informed Neural Networks (PINNs)**. Master of Science Thesis in Physics,

Graduate School, Khon Kaen University.

Thesis Advisor: Assoc. Prof. Dr. Chakrit Pongkitivanichkul

ABSTRACT

Physics-Informed Neural Networks (PINNs) are a type of neural network designed to solve problems governed by physical laws. Instead of relying solely on data, PINNs include physics equations, such as differential equations, directly in their training process. This approach allows the model to find solutions that are both numerically accurate and physically consistent. In this work, we apply PINNs to axion cosmology in order to increase the accuracy and stability of training. We use Fourier feature embeddings to capture oscillatory behaviors and residual blocks to reduce vanishing gradients. Our results indicate that the PINNs framework can successfully reproduce the evolution of the axion scalar field with accuracy, showing excellent agreement with traditional numerical solutions. These findings highlight the potential of PINNs as a promising alternative for solving complex differential equations. Moreover, the method shows potential for future development and extension to multi-field systems or other non-linear dynamical problems in modern theoretical physics.

To my family and entire teaching staff.

ACKNOWLEDGEMENTS

I would like to express my gratitude to many people who involve in this work. I want to thank my advisor, Assoc. Prof. Dr. Chakrit Pongkitivanichkul for their support through many hours of enlightening meetings and discussions. I would like to thank my senior, Grit Saowanit who helped solve problem and gave advices.

We also would like to thank members of the KKPACT group for useful discussions.

Moreover, I would like to thank the Department of Physics and the Faculty of Science at Khon Kaen University for their teaching. I thank all the lecturers who have trained and taught me at KKU.

This acknowledgement would be incomplete without mentioning my family and friends. I would like to thank to my parents, my sister, my brother and friends for encouraging me and for their unconditional support.

Weerachai Seeonsaeng

TABLE OF CONTENTS

	Page
ABSTRACT (IN THAI)	i
ABSTRACT (IN ENGLISH)	ii
DEDICATION	iii
ACKNOWLEDGEMENTS	iv
LIST OF FIGURES	vii
CHAPTER I INTRODUCTION	1
CHAPTER II BACKGROUND COSMOLOGY	3
2.1 The Expanding Universe	3
2.2 Friedmann–Lemaître–Robertson–Walker Metric (FLRW)	8
2.3 Redshift and Hubble law	10
2.4 Friedmann Equation	13
2.5 Λ CDM: Cosmological Model with Dark Energy and Dark Matter	25
2.6 Evolution of scalar fields	29
CHAPTER III NEURAL NETWORKS	35
3.1 Training and Optimization	41
3.2 Optimization via Gradient Descent	43
3.3 The Backpropagation Algorithm	45
3.4 Types of Neural Networks	47
3.5 Regularization in Neural Networks	48
3.6 Challenges in Neural Network Training	49
3.7 Applications and Transition to Physics-Informed Neural Networks	50
CHAPTER IV PHYSICS-INFORMED NEURAL NETWORKS (PINNs)	51
4.1 Origin of Physics-Informed Neural Networks	51
4.2 Structure of Physics-Informed Neural Networks	53
4.3 Application of PINNs to Axion Cosmology	55
4.4 Integrating Fourier Features and Residual Block into PINNs	58

TABLE OF CONTENTS (Cont.)

	Page
4.5 ODEs Solver	59
4.6 Hyperparameter tuning	59
CHAPTER V RESULTS AND DISCUSSION	60
5.1 PINNs without Fourier features and residual blocks	60
5.2 PINNs with Fourier features and residual blocks	63
CHAPTER VI SUMMARY AND OUTLOOK	73
REFERENCES	75
APPENDICES	80
APPENDIX A Programming Environment	80
APPENDIX B Code for PINNs model	81
APPENDIX C Code for hyperparameter tuning	93
APPENDIX D Code for contour plot	95

LIST OF FIGURES

	Page
Figure 2.1	8
Figure 2.2	28
Figure 3.1	35
Figure 3.2	37
Figure 3.3	38
Figure 3.4	39
Figure 3.5	40
Figure 3.6	41
Figure 4.1	53
Figure 4.2	55
Figure 4.3	59
Figure 5.1	60
Figure 5.2	61
Figure 5.3	61

LIST OF FIGURES (Cont.)

		Page
Figure 5.4	The evolution of leaning curve during training with Adam optimizer for baseline models trained on scalar field dynamics with scalar field mass $m_a = 10$.	62
Figure 5.5	Contour plots of the mean absolute error (MAE) for $\phi(t)$ with scalar field mass $m_a = 10$. This is the hyperparameter value obtained from scanning the MAE values for λ_{IC} and λ_{phys} , which yielded the minimum values of the hyperparameters are approximately 6.05×10^{-3} occurs near $\lambda_{\text{IC}} = 190$, $\lambda_{\text{phys}} = 5$.	62
Figure 5.6	Contour plots of the mean absolute error (MAE) for $\phi(t)$ with scalar field mass $m_a = 10$. The MAE of λ_a and λ_ϕ at the minimum point approximately 6.83×10^{-4} at $\lambda_a = 3$, $\lambda_\phi = 8$.	63
Figure 5.7	Comparison between ODEs and PINNs for $\phi(t)$ with scalar field mass $m_a = 10$ after adding Fourier features and residual blocks.	64
Figure 5.8	Comparison between ODEs and PINNs for $a(t)$ with scalar field mass $m_a = 10$. It was found that the results after adding Fourier features and residual blocks were much more different from the ODEs.	64
Figure 5.9	Comparison between ODEs and PINNs for $a(t)$ in linear scale with scalar field mass $m_a = 10$. This graph presents the comparison of the scale factor $a(t)$ in logarithmic scale. The consistency is found to remain good at late times, but not as good as the case without Fourier features and residual blocks. The logarithmic graph shows that the deviations that appeared in the early universe are still very different.	65

LIST OF FIGURES (Cont.)

	Page
Figure 5.10 The evolution of leaning curve during training with Adam optimizer for baseline models trained on scalar field dynamics with scalar field mass $m_a = 10$. We can see that the value of the learning curve has increased more than before adding Fourier features and residual blocks.	65
Figure 5.11 Comparison between ODEs and PINNs for $\phi(t)$ (oscillation line) and $a(t)$ (exponential line) with scalar field mass $m_a = 30$.	66
Figure 5.12 The evolution of leaning curve during training with Adam optimizer for baseline models trained on scalar field dynamics with scalar field mass $m_a = 30$.	66
Figure 5.13 Contour plots of the mean absolute error (MAE) for $\phi(t)$ with scalar field mass $m_a = 10$. This is the hyperparameter value obtained from scanning the MAE values for λ_{IC} and λ_{phys} , which yielded the minimum values of the hyperparameters are approximately 9.90×10^{-4} occurs near $\lambda_{IC} = 150$, $\lambda_{phys} = 12$.	67
Figure 5.14 Contour plots of the mean absolute error (MAE) for $\phi(t)$ with scalar field mass $m_a = 30$. The MAE of λ_a and λ_ϕ at the minimum point approximately 2.34×10^{-3} at $\lambda_a = 5.5$, $\lambda_\phi = 35.0$.	67
Figure 5.15 Comparison between ODEs and PINNs for $\phi(t)$ (oscillation line) and $a(t)$ (exponential line) with scalar field mass $m_a = 40$, using the same hyperparameter as $m_a = 30$.	68
Figure 5.16 The evolution of leaning curve during training with Adam optimizer for baseline models trained on scalar field dynamics with scalar field mass $m_a = 40$, using the same hyperparameter as $m_a = 30$.	68

LIST OF FIGURES (Cont.)

	Page
Figure 5.17 Comparison between ODEs and PINNs for $\phi(t)$ (oscillation line) and $a(t)$ (exponential line) with scalar field mass $m_a = 50$, using the same hyperparameter as $m_a = 30$.	69
Figure 5.18 The evolution of leaning curve during training with Adam optimizer for baseline models trained on scalar field dynamics with scalar field mass $m_a = 50$, using the same hyperparameter as $m_a = 30$.	69
Figure 5.19 Comparison between ODEs and PINNs for $\phi(t)$ (oscillation line) and $a(t)$ (exponential line) with scalar field mass $m_a = 60$, using the same hyperparameter as $m_a = 30$.	70
Figure 5.20 The evolution of leaning curve during training with Adam optimizer for baseline models trained on scalar field dynamics with scalar field mass $m_a = 60$, using the same hyperparameter as $m_a = 30$.	70
Figure 5.21 Comparison between ODEs and PINNs for $\phi(t)$ (oscillation line) and $a(t)$ (exponential line) with scalar field mass $m_a = 70$.	71
Figure 5.22 The evolution of leaning curve during training with Adam optimizer for baseline models trained on scalar field dynamics with scalar field mass $m_a = 70$.	71
Figure 5.23 Contour plots of the mean absolute error (MAE) for $\phi(t)$ with scalar field mass $m_a = 70$. This is the hyperparameter value obtained from scanning the MAE values for λ_{IC} and λ_{phys} , which yielded the minimum values of the hyperparameters are approximately 2.75×10^{-2} occurs near $\lambda_{IC} = 300$, $\lambda_{phys} = 10$. We re-evaluated the hyperparameter values to find the best MAE fro $m_a = 70$.	72

LIST OF FIGURES (Cont.)

	Page
Figure 5.24	
Contour plots of the mean absolute error (MAE) for $\phi(t)$ with scalar field mass $m_a = 70$. The MAE of λ_a and λ_ϕ at the minimum point approximately 8.40×10^{-3} at $\lambda_a = 4, \lambda_\phi = 30$.	72

CHAPTER I

INTRODUCTION

The evolution of scalar fields plays a central role in many areas of cosmology, from inflation to dark matter models involving axion-like particles. In this context, axions are pseudo-scalar fields that arise naturally in solutions to the strong CP problem in QCD and as generic predictions in string theory and other high-energy frameworks. From a cosmological viewpoint, axions evolve as coherent classical fields in the expanding universe, governed by the Klein–Gordon equation in the Friedmann–Lemaître–Robertson–Walker (FLRW) background. Initially frozen by Hubble friction, the axion field remains nearly constant until the expansion rate drops below the axion mass scale ($H \sim m_a$), at which point it begins oscillating around the minimum of its potential. These oscillations act like non-relativistic matter, with the axion energy density scaling as $\rho_a \propto a^{-3}$, making axions natural candidates for cold dark matter. Their presence affects the evolution of the universe from the epoch of inflation through structure formation and the cosmic microwave background, connecting fundamental particle physics to observable cosmological phenomena [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12].

Solving their equations of motion typically requires numerically integrating stiff ordinary differential equations (ODEs), especially due to the rapid transition in axion behavior as the universe evolves. In the early universe, when the Hubble parameter $H \gg m_a$, the axion field is overdamped by cosmic expansion and remains effectively frozen at its initial value this is the “frozen phase”, where its potential energy dominates and the field behaves like a cosmological constant. As the universe expands and the Hubble rate decreases to $H \sim m_a$, the field enters a transition phase, during which it begins to roll down and oscillate around the minimum of its potential. These coherent oscillations mark the onset of the “oscillatory phase”, where the behavior of the axion shifts to that of a pressureless fluid ($P = 0$), making it a viable cold dark matter candidate with the scaling of the energy density as $\rho_a \propto a^{-3}$. In some scenarios, especially when their mass evolves with time, the dynamics

become highly-nonlinear and require careful numerical integration of stiff, coupled equations.

However, as the number of fields increases—such as in N -axion scenarios—the computational cost grows significantly. As the number of axion fields increases particularly in scenarios involving 10 – 100 of fields the computational complexity of modeling their cosmological evolution escalates significantly. This is due to the necessity of solving a large system of coupled, stiff ordinary differential equations (ODEs) [5].

Physics-Informed Neural Networks (PINNs) [13, 14, 15, 16, 17, 18] have emerged as a mesh-free method to approximate solutions to differential equations by embedding the physical laws directly into the loss function. Their flexibility has enabled applications across a wide range of physics domains. In classical mechanics, PINNs are used to recover motion governed by Newtonian or Lagrangian dynamics, while in general relativity they are applied to solve Einstein’s field equations [15, 19, 20]. This broad applicability demonstrates the strength of PINNs in unifying machine learning with fundamental physics, especially for systems where data is scarce but governing equations are well known.

In this study, we conduct a preliminary comparison between Physics-Informed Neural Networks (PINNs) and conventional ODE solvers for a simple single-field case. The objective is to assess the performance of PINNs and to explore training strategies that can be readily extended to more complex multi-field systems.

CHAPTER II

BACKGROUND COSMOLOGY

Based on general relativity, the modern standard model of cosmology, known as the Λ CDM model, is used to describe the Universe as expanding from an initial hot and dense state known as the Big Bang. The expansion is described by the scale factor $a(t)$, which evolves through various cosmic epochs: radiation-dominated, matter-dominated, and dark energy-dominated. The dynamics of this expansion are governed by the Friedmann equations, which relate the Hubble parameter $H(t)$ to the total energy density of the universe.

2.1 The Expanding Universe

One of the foundational assumptions in modern cosmology is the cosmological principle, which asserts that the universe is homogeneous and isotropic on sufficiently large scales. Homogeneity means that the universe has the same physical properties at every location, while isotropy means that it appears the same in all directions when observed from any point. Although local structures such as galaxies, clusters, and voids break this symmetry on small scales, the large-scale distribution of matter and energy supports the validity of the cosmological principle. This assumption significantly simplifies our understanding of the universe by allowing us to model the spatial geometry at a fixed cosmic time as a homogeneous and isotropic three-dimensional manifold. As a result, general relativity constrains the geometry of spacetime to one of three possible spatial curvatures: positive curvature (a 3-sphere or closed universe), flat space (Euclidean space or flat universe), and negative curvature (a 3-hyperboloid or open universe). These three cases correspond to constant spatial curvature. These geometries can also be visualized more intuitively by considering their embeddings in higher-dimensional Euclidean space:

Flat space: The simplest form of a homogeneous and isotropic three-dimensional space is flat space, also known as Euclidean space, denoted as $\mathbf{R}^3$. In this geometry,

the curvature is zero, and the spatial structure extends infinitely in all directions without bending. The geometry of such a space is characterized by a metric, which provides a mathematical rule for determining the distance between two points. In particular, the line element ds specifies the infinitesimal distance between nearby points and defines the flat space metric as,

$$ds^2 = dx^2 + dy^2 + dz^2. \quad (2.1)$$

In addition to standard coordinates, we will also consider other systems like spherical polar coordinates,

$$x = r \sin\theta \cos\phi, \quad y = r \sin\theta \sin\phi, \quad z = r \cos\theta. \quad (2.2)$$

We calculate the metric in these coordinates by translating small variations in (r, θ, ϕ) into Cartesian coordinates (x, y, z) using the Leibniz rule, which yields,

$$\begin{aligned} dx &= dr \sin\theta \cos\phi + r \cos\theta \cos\phi d\theta - r \sin\theta \sin\phi d\phi, \\ dy &= dr \sin\theta \sin\phi + r \cos\theta \sin\phi d\theta + r \sin\theta \cos\phi d\phi, \\ dz &= dr \cos\theta - r \sin\theta d\theta. \end{aligned}$$

Substituting these expressions into the flat metric (2.1) gives us the flat metric in polar coordinates,

$$ds^2 = dr^2 + r^2(d\theta^2 + \sin^2\theta d\phi^2). \quad (2.3)$$

Positive curvature, as one of the possible spatial geometries consistent with homogeneity and isotropy, is represented by the three-dimensional sphere $\mathbf{S}^3$, which represents a space with positive curvature. This geometry can be intuitively understood through its embedding in four-dimensional Euclidean space $\mathbf{R}^4$. In this context, the 3-sphere is defined by the standard equation,

$$x^2 + y^2 + z^2 + w^2 = R^2,$$

where R is the curvature radius of three spheres and (x, y, z, w) are the hypersurface represents all points at a fixed distance R from the origin in four-dimensional space, analogously to how a 2-sphere (the surface of a ball) is embedded in three-dimensional

space. Such a space provides a compact, closed universe model in cosmology and plays a fundamental role in describing positively curved FLRW universes.

There are once again multiple ways to choose coordinate systems. One approach involves keeping the 3D spherical polar coordinates (2.2) and eliminating the coordinate w via the relation $w^2 = R^2 - r^2$. In this formulation, a point on the 3-sphere $\mathbf{S}^3$ is described by a radial coordinate $r \in [0, R]$, along with angular coordinates $\theta \in [0, \pi]$ and $\phi \in [0, 2\pi)$. The metric on $\mathbf{S}^3$ can then be derived by observing that,

$$w^2 = R^2 - r^2 \rightarrow dw = -\frac{r dr}{\sqrt{R^2 - r^2}}.$$

The geometry of the 3-sphere $\mathbf{S}^3$ is derived from the induced metric of the flat space $\mathbf{R}^4$. By substituting the parameterization into the flat metric $ds^2 = dx^2 + dy^2 + dz^2 + dw^2$, we obtain the corresponding metric on $\mathbf{S}^3$,

$$ds^2 = \frac{R^2}{R^2 - r^2} dr^2 + r^2(d\theta^2 + \sin^2\theta d\phi^2). \quad (2.4)$$

Arguably, a more natural choice of coordinates arises from the 4-dimensional generalization of the spherical polar coordinates given in (2.2), which can be introduced by defining $r = R \sin\chi$. By introducing the spherical coordinates (χ, θ, ϕ) , the full coordinate system becomes,

$$x = R \sin\chi \sin\theta \cos\phi,$$

$$y = R \sin\chi \sin\theta \sin\phi,$$

$$z = R \sin\chi \cos\theta,$$

$$w = R \cos\chi.$$

Then the distance between two points on a three-sphere with coordinates (χ, θ, ϕ) and $(\chi + d\chi, \theta + d\theta, \phi + d\phi)$ is,

$$\begin{aligned} ds^2 &= dx^2 + dy^2 + dz^2 + dw^2, \\ &= R^2 \{ [d(\sin\chi \sin\theta \cos\phi)]^2 + [d(\sin\chi \sin\theta \sin\phi)]^2 + [d(\sin\chi \cos\theta)]^2 \\ &\quad + [d(\cos\chi)]^2 \}. \end{aligned}$$

After calculation, we find that the line element (metric) on the three spheres becomes,

$$ds^2 = R^2 [d\chi^2 + \sin^2\chi (d\theta^2 + \sin^2\theta d\phi^2)]. \quad (2.5)$$

Negative curvature corresponds to the last homogeneous and isotropic space we consider, which is perhaps the least familiar. It is the hyperboloid $\mathbf{H}^3$, which can also be defined through an embedding in $\mathbf{R}^4$, this time with the relation,

$$x^2 + y^2 + z^2 - w^2 = -R^2, \quad (2.6)$$

or,

$$w^2 - x^2 - y^2 - z^2 = R^2. \quad (2.7)$$

The coordinate on a three-hyperboloid can be introduced by,

$$x = R \sinh\chi \sin\theta \cos\phi,$$

$$y = R \sinh\chi \sin\theta \sin\phi,$$

$$z = R \sinh\chi \cos\theta,$$

$$w = R \cosh\chi.$$

This space exhibits uniform negative curvature, where parallel lines diverge a hallmark of hyperbolic geometry. As before, the metric is derived from an embedding in $\mathbf{R}^4$, but with a different signature, namely $(-, +, +, +)$. Consequently, the flat metric takes the form $ds^2 = -dw^2 + dx^2 + dy^2 + dz^2$, consistent with the embedding described in equation (2.6). By adopting the 3D coordinate system (r, θ, ϕ) , the relation $w^2 = r^2 + R^2$ holds, and the resulting metric becomes,

$$ds^2 = \frac{R^2}{R^2 + r^2} dr^2 + r^2(d\theta^2 + \sin^2\theta d\phi^2). \quad (2.8)$$

We can write $r = R \sinh\chi$, the distance between two points can be determined using the following formula,

$$ds^2 = R^2[d\chi^2 + \sinh^2\chi(d\theta^2 + \sin^2\theta d\phi^2)]. \quad (2.9)$$

To unify these cases, we can define a general metric,

$$ds^2 = d\rho^2 + r^2(\rho)(d\theta^2 + \sin^2\theta d\phi^2), \quad (2.10)$$

where

$$r(\rho) = \begin{cases} R \sin(\rho/R) & \text{spherical space,} \\ \rho & \text{flat space,} \\ R \sinh(\rho/R) & \text{hyperbolic space.} \end{cases}$$

A homogeneous and isotropic space has the characteristic that all covariant geometric quantities can be expressed in terms of the metric tensor γ_{ij} , and may also involve the Kronecker delta δ_i^j and the Levi-Civita symbol E_{ijk} , both of which can exist in any Riemannian manifold. Moreover, the components of these tensors are independent of the spatial coordinates. Specifically, the Riemann curvature tensor in such a space is given by,

$$R_{ijkl} = \frac{\kappa}{R^2}(\gamma_{ik}\gamma_{jl} - \gamma_{il}\gamma_{jk}), \quad (2.11)$$

where parameter $\kappa = 0, \pm 1$, which distinguishes three-space,

$$\kappa = \begin{cases} +1, & \text{spherical space (close),} \\ 0, & \text{flat space (flat),} \\ -1, & \text{hyperbolic space (open).} \end{cases}$$

Follows from (2.11) the Ricci tensor is,

$$R_{ij} = 2\frac{\kappa}{R^2}\gamma_{ij}. \quad (2.12)$$

The spatial curvature scalar is constant and given by $6\kappa/R^2$. The interpretation of the quantities in equation (2.10) is straightforward: ρ represents the (shortest) geodesic distance from the origin to a point with coordinates (ρ, θ, ϕ) , while $r(\rho)$ determines the area of the two-dimensional sphere located at that distance from the origin, with $S = 4\pi r^2(\rho)$. Furthermore, it is evident that a length interval l , located at a distance ρ from the origin, subtends a specific angle as seen from the origin,

$$\Delta\theta = \frac{l}{r(\rho)}.$$

We can choose r instead of ρ as the radial coordinate. From this we obtain for a hyperbola,

$$\begin{aligned} d\rho^2 &= \frac{dr^2}{\cosh(\rho/R)}, \\ &= \frac{dr^2}{1 + \frac{r^2}{R^2}}. \end{aligned}$$

The metrics of three-space can be written as,

$$ds^2 = \frac{dr^2}{1 - \kappa \frac{r^2}{R^2}} + r^2(d\theta^2 + \sin^2\theta d\phi^2), \quad (2.13)$$

for the coordinates (ρ, θ, ϕ) cover only half of the three-sphere: in range of $1 \leq r < R$ is part of the three-sphere extending from the origin.

2.2 Friedmann–Lemaître–Robertson–Walker Metric (FLRW)

An expanding, homogeneous, and isotropic Universe is described by the metric

$$ds^2 = dt^2 - a^2(t)\gamma_{ij}dx^i dx^j, \quad (2.14)$$

or equivalently

$$ds^2 = dt^2 - a^2(t) \left[\frac{dr^2}{1 - \kappa \frac{r^2}{R^2}} + r^2(d\theta^2 + \sin^2\theta d\phi^2) \right], \quad (2.15)$$

where γ_{ij} is the diagonal metric of three-sphere, this metric is called Friedmann–Lemaître–Robertson–Walker Metric (FLRW). The function $a(t)$, known as the scale factor, represents the radius of spatial curvature at a given moment in time.

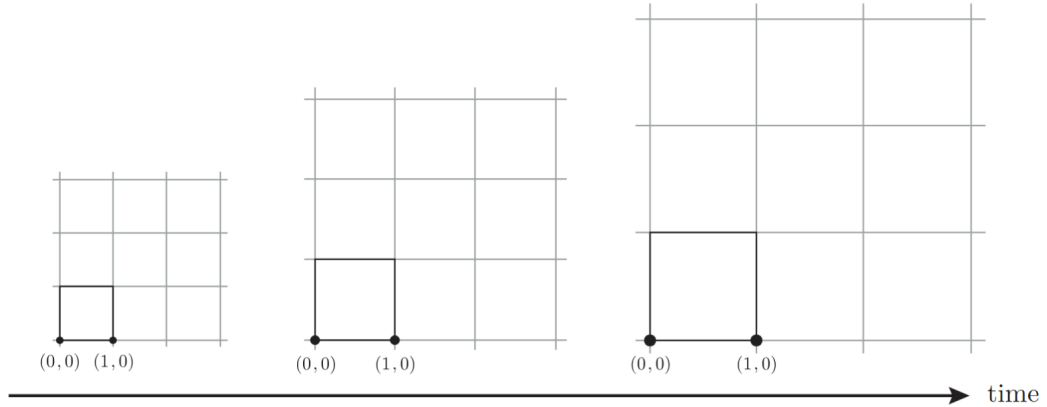


Figure 2.1 The expansion of the universe. The physical distance between fixed co-moving coordinates increases with time. [21]

In a spatially flat Universe, the absolute value of the scale factor lacks intrinsic physical significance, as it can be arbitrarily normalized—commonly by setting $a_0 = a(t_0) = 1$ through a rescaling of spatial coordinates. What is physically meaningful, however, is the relative scaling between different epochs, characterized by the ratio $a(t_1)/a(t_2)$. Of particular importance is the Hubble parameter $H(t)$, which quantifies

the rate of cosmic expansion,

$$H(t) = \frac{\dot{a}(t)}{a(t)}.$$

According to the coordinate conventions used in closed and open universes, the spatial coordinates x^i are dimensionless, while the scale factor $a(t)$ carries the dimension of length. In contrast, for a spatially flat universe, it is often more convenient to assign spatial coordinates x^i the dimension of length and treat $a(t)$ as dimensionless. The metric in equation 2.14 is expressed in a comoving frame, where the Universe appears homogeneous at every moment in time and the matter distribution is at rest—neglecting local peculiar motions such as galaxies orbiting within clusters. In this frame, the worldlines $x^i = \text{const}$ represent geodesics, indicating that particles initially at rest move along free-fall trajectories. For such particles, the proper time equals the coordinate time, i.e., $ds^2 = dt^2$. This property naturally selects the co-moving frame as the preferred coordinate system for describing cosmological evolution. To confirm this, we now examine the geodesic equation,

$$\frac{d}{ds}u^\mu + \Gamma_{\nu\lambda}^\mu u^\nu u^\lambda = 0, \quad (2.16)$$

where u^μ is four-velocity. Accordingly, we now calculate the Christoffel symbols,

$$\Gamma_{\nu\lambda}^\mu = \frac{1}{2}g^{\mu\sigma}(\partial_\nu g_{\lambda\sigma} + \partial_\lambda g_{\nu\sigma} - \partial_\sigma g_{\nu\lambda}), \quad (2.17)$$

and

$$\partial_\mu = \partial/\partial x^\mu.$$

The components of the metric tensor that are nonzero are given as follows,

$$g_{00} = 1, \quad g_{ij} = -a^2(t)\gamma_{ij}(x),$$

and the inverse tensor is

$$g^{00} = 1, \quad g^{ij} = \frac{1}{-a^2(t)}\gamma^{ij}(x).$$

Obviously,

$$\Gamma_{00}^0 = \Gamma_{0i}^0 = \Gamma_{00}^i = 0,$$

in the expression in parentheses in (2.17), at least two indices are equal to zero, but $g_{0i} = 0, \partial_\mu g_{00} = 0$). For Γ_{0j}^i we have,

$$\Gamma_{0j}^i = \frac{1}{2} g^{ik} \partial_0 g_{jk} = \frac{\dot{a}}{a} \delta_j^i.$$

The remaining Christoffel symbols can be readily obtained from the metric by applying the standard formula,

$$\begin{aligned} \Gamma_{ij}^0 &= a\dot{a}\gamma_{ij}, \\ \Gamma_{jk}^i &= {}^{(3)}\Gamma_{jk}^i. \end{aligned}$$

For a co-moving particle,

$$u^0 = \frac{dx^0}{ds} = \frac{dt}{dt} = 1, \quad u^i = 0,$$

therefore,

$$\frac{du^\mu}{ds} = 0.$$

Since $\Gamma_{00}^\mu = 0$ for all μ , the geodesic equation is satisfied. To conclude this section, it is worth noting that in both closed and open cosmological models, spatial curvature is often negligible. In such cases, the spatially flat metric provides a good approximation and is commonly adopted,

$$\gamma_{ij} = \delta_{ij}. \tag{2.18}$$

This approximation is certainly valid for physical processes occurring on spatial scales significantly smaller than the curvature radius $a(t)$. As previously noted in this section, the Universe is, to a very high degree of accuracy, spatially flat, both in the early Universe and at present. Therefore, the use of approximation (2.18) is well justified across a wide range of scales. A more precise assessment of this claim will be provided in the subsequent discussion.

2.3 Redshift and Hubble law

As the scale factor $a(t)$ increases with time, the physical distance between points with fixed comoving spatial coordinates x^i also increases — indicating that

the Universe is expanding. Consequently, the wavelength of a photon emitted by a distant source increases as the photon travels toward the observer; in other words, the photon experiences a redshift. This cosmological redshift phenomenon can be formally described by considering the action of a free electromagnetic field propagating in a curved spacetime with metric $g_{\mu\nu}(x)$,

$$S = -\frac{1}{4} \int d^4x \sqrt{-g} g^{\mu\nu} g^{\lambda\rho} F_{\mu\lambda} F_{\nu\rho}. \quad (2.19)$$

Normally,

$$F_{\mu\nu} = \nabla_\mu A_\nu - \nabla_\nu A_\mu = \partial_\mu A_\nu - \partial_\nu A_\mu, \quad (2.20)$$

here A_μ denotes the electromagnetic four-potential. The expression in Equation (2.19) represents the most straightforward covariant extension of Maxwell's classical electrodynamics in curved spacetime. To study photon propagation within a homogeneous and isotropic universe, we note that the typical photon wavelength is much smaller than the spatial curvature radius of the universe, regardless of whether the universe is spatially open or closed. Consequently, it is appropriate to approximate the universe as spatially flat and to adopt metric units in our analysis.

$$ds^2 = dt^2 - a^2(t) \delta_{ij} dx^i dx^j. \quad (2.21)$$

When we use conformal time η instead of cosmic time t , defined by

$$dt = a d\eta, \quad \eta = \int \frac{dt}{a(t)}. \quad (2.22)$$

We can rewrite the new time coordinates metric as follows,

$$ds^2 = a^2(\eta) [d\eta^2 - \delta_{ij} dx^i dx^j]. \quad (2.23)$$

In other words,

$$g_{\mu\nu} = a^2(\eta) \eta_{\mu\nu}, \quad (2.24)$$

where $\eta_{\mu\nu}$ denote the Minkowski metric. This metric differs from the Minkowski metric in that the overall rescaling depends on time, i.e., the metric of a homogeneous

isotropic universe has a flat conformal form in coordinates (η, x^i) . In these coordinates, we have

$$g^{\mu\nu} = a^{-2}\eta^{\mu\nu}, \quad (2.25)$$

and

$$g = \det(g_{\mu\nu}), \quad (2.26)$$

which gives

$$\sqrt{-g} = a^4. \quad (2.27)$$

By substituting into action for free electromagnetic field in space-time we obtain that in coordinates (η, x^i) coincides with,

$$S = -\frac{1}{4} \int d^4x \eta^{\mu\nu} \eta^{\lambda\sigma} F_{\mu\lambda} F_{\nu\rho}. \quad (2.28)$$

Such characteristics are specific to the theory of massless vector fields. In contrast, for other types of vector field theories, the action expressed in conformal coordinates (η, x^i) typically does not simplify to the form it takes in flat spacetime. Therefore, the free electromagnetic field is classified as conformal, whereas most other fields are considered non-conformal. From the preceding equation, it follows that the solutions to the equations governing the free electromagnetic field in a universe described by the metric (2.21) can be represented as superpositions of plane waves,

$$A_\mu^{(\alpha)} = e_\mu^{(\alpha)} e^{ik\eta - i\mathbf{k}\mathbf{x}}, \quad (2.29)$$

where $\mathbf{k}$ is a constant vector (called conformal momentum, or coordinate momentum), $k = |\mathbf{k}|$, and $e_\mu^{(\alpha)}$ is the standard polarization vectors of a photon, $\alpha = 1, 2$. We emphasize that $\mathbf{k}$ is not the physical momentum of the photon, and k is not the physical frequency, since dx and $d\eta$ are not physical distances and physical time periods, the quantity $\Delta x = 2\pi/k$ is the coordinate wavelength of the photon, while the physical wavelength at time t according to (2.21) is

$$\lambda(t) = a(t)\Delta x = 2\pi \frac{a(t)}{k}. \quad (2.30)$$

Likewise, $\Delta\eta = 2\pi/k$ is the period of electromagnetic wave in conformal time, the period in physical time² t is

$$T = a(t)\Delta\eta = 2\pi\frac{a(t)}{k}, \quad (2.31)$$

and the physical momentum $\mathbf{p}$ and physical frequency of a photon at time t are

$$\mathbf{p}(t) = \frac{\mathbf{k}}{a(t)}, \quad \omega(t) = \frac{k}{a(t)}. \quad (2.32)$$

For an expanding Universe, the scaling factor $a(t)$ depend on time, the physical wavelength increase, while the physical momentum and frequency decrease: they are redshifted. If a photon is emitted at time t_i with a given wavelength λ_i (as in the transition of a hydrogen atom from an excited to a fundamental state), the observed wavelength at Earth is given by

$$\lambda_0 = \lambda_i \frac{a_0}{a(t_i)} \equiv \lambda_i [1 + z(t_i)]. \quad (2.33)$$

Normally, the index 0 means the current value of the related quantity. The relationship of redshift can be written as

$$z(t) = \frac{a_0}{a(t)} - 1, \quad (2.34)$$

where $z(t)$ is redshift

2.4 Friedmann Equation

The cosmological expansion emerges as a direct consequence of the dynamics of spacetime governed by Einstein field equations. These equations relate the geometry of spacetime encoded in the metric tensor $g_{\mu\nu}$ to distribution and evolution of energy and matter through the stress-energy tensor $T_{\mu\nu}$. In the context of a homogeneous and isotropic universe, the spacetime metric is given by the Friedmann–Lemaître–Robertson–Walker Metric (FLRW) form, in which the scale factor $a(t)$ characterizes the relative size of the universe as a function of cosmic time t . Given by Einstein field equations,

$$R_{\mu\nu} - \frac{1}{2}g_{\mu\nu}R = 8\pi GT_{\mu\nu}. \quad (2.35)$$

We begin with the calculation of the Ricci tensor $R_{\mu\nu}$,

$$R_{\mu\nu} = \partial_\lambda \Gamma_{\mu\nu}^\lambda - \partial_\mu \Gamma_{\nu\lambda}^\lambda + \Gamma_{\mu\nu}^\lambda \Gamma_{\lambda\sigma}^\sigma - \Gamma_{\mu\sigma}^\lambda \Gamma_{\lambda\nu}^\sigma, \quad (2.36)$$

where Γ_{jk}^i is Christoffel symbol. The Christoffel symbol can be written according to the following relation,

$$\Gamma_{\alpha\beta}^\mu = \frac{1}{2} g^{\mu\nu} (\partial_\beta g_{\alpha\nu} + \partial_\alpha g_{\beta\nu} - \partial_\nu g_{\alpha\beta}). \quad (2.37)$$

The Christoffel symbols also appear in the geodesic equation, which describes the path that a free-falling particle (or light ray) follows in curved spacetime. In general relativity, geodesics represent the generalization of straight lines to curved geometry, and are determined by the equation,

$$\frac{d^2 x^\mu}{d\tau^2} + \Gamma_{\alpha\beta}^\mu \frac{dx^\alpha}{d\tau} \frac{dx^\beta}{d\tau} = 0, \quad (2.38)$$

where τ is the affine parameter along the trajectory — proper time for massive particles, and an arbitrary parameter for massless particles.

This equation encodes the influence of spacetime curvature (via $\Gamma_{\alpha\beta}^\mu$) on the motion of particles. In the context of cosmology, solving this equation for the FLRW metric yields the behavior of particle trajectories, including light rays (null geodesics), which are essential for calculating cosmological distances, redshift, and horizons.

The non-vanishing components of the Christoffel symbol. These formulas specifically indicate that

$$\Gamma_{0\mu}^\mu = \frac{\dot{a}}{a} \delta_\mu^i = 3 \frac{\dot{a}}{a}, \quad (2.39)$$

$$\Gamma_{i\mu}^\mu = {}^{(3)}\Gamma_{ij}^j. \quad (2.40)$$

Let calculate R_{00} . Since $\Gamma_{00}^\mu = 0$, the only contributions come from the second and fourth terms, and we obtain,

$$R_{00} = -\partial_0 \Gamma_{0\lambda}^\lambda - \Gamma_{0\sigma}^\lambda \Gamma_{0\lambda}^\sigma \quad (2.41)$$

$$= -\partial_0 \Gamma_{0\lambda}^\lambda - \Gamma_{0j}^i \Gamma_{0i}^j \quad (2.42)$$

$$= -\partial_0 \left(3 \frac{\dot{a}}{a} \right) - \left(\frac{\dot{a}}{a} \right)^2 \delta_j^i \delta_i^j \quad (2.43)$$

$$= -3\partial_0 \left(\frac{\dot{a}}{a} \right) - 3 \left(\frac{\dot{a}}{a} \right)^2 \quad (2.44)$$

$$= -3 \frac{\ddot{a}}{a}. \quad (2.45)$$

Now let's look at the mixed components R_{0i} ,

$$R_{0i} = \partial_j \Gamma_{0i}^j - \partial_0 \Gamma_{i\lambda}^\lambda + \Gamma_{0i}^j \Gamma_{j\lambda}^\lambda - \Gamma_{0j}^k \Gamma_{ik}^j \quad (2.46)$$

$$= 0. \quad (2.47)$$

The finally calculate the spatial components R_{ij} ,

$$R_{ij} = (\partial_0 \Gamma_{ij}^0 - \partial_k \Gamma_{ij}^k) - \partial_i \Gamma_{j\lambda}^\lambda + (\Gamma_{ij}^0 \Gamma_{0\sigma}^\sigma + \Gamma_{ij}^k \Gamma_{k\sigma}^\sigma) - \quad (2.48)$$

$$(\Gamma_{ik}^0 \Gamma_{j0}^k + \Gamma_{i0}^k \Gamma_{jk}^0 + \Gamma_{il}^k \Gamma_{jk}^l) \quad (2.49)$$

$$= \partial_0(\dot{a}a)\gamma_{ij} + \dot{a}a\gamma_{ij} \cdot 3\frac{\dot{a}}{a} - \dot{a}a\gamma_{ik} \cdot \frac{\dot{a}}{a}\delta_j^k - \frac{\dot{a}}{a}\delta_i^k \dot{a}a\gamma_{jk} + {}^{(3)}R_{ij} \quad (2.50)$$

$$= (\ddot{a}a + 2\dot{a}^2 + 2\kappa)\gamma_{ij}. \quad (2.51)$$

By substituting (2.41), (2.46), and (2.48) into Ricci tensor to find the scalar curvature,

$$R = g^{\mu\nu} R_{\mu\nu} \quad (2.52)$$

$$= g^{00} R_{00} + g^{ij} R_{ij} \quad (2.53)$$

$$= R_{00} - \frac{1}{a^2} \gamma^{ij} R_{ij} \quad (2.54)$$

$$= -6 \left(\frac{\ddot{a}}{a} + \frac{\dot{a}^2}{a^2} + \frac{\kappa}{a^2} \right), \quad (2.55)$$

where

$$\gamma^{ij}\gamma_{ij} = 3$$

.

The result from component (00) of the left-hand side of Einstein equation is,

$$R_{00} - \frac{1}{2}g_{00}R = 3 \left(\frac{\dot{a}^2}{a^2} + \frac{\kappa}{a^2} \right). \quad (2.56)$$

The other components of the Einstein tensor $G_{\mu\nu} \equiv R_{\mu\nu} - \frac{1}{2}g_{\mu\nu}R$ are calculated in a same way.

Let us now examine the right-hand side of the Einstein equations. In the cosmological epoch of interest, the matter content of the Universe can be modeled, on large scales, as a homogeneous and isotropic perfect fluid. This approximation is justified by observations indicating that the Universe is highly uniform on large scales.

A perfect fluid is described by the stress-energy tensor (also known as the energy-momentum tensor), which takes the form,

$$T_{\mu\nu} = (\rho + p)u_\mu u_\nu - g_{\mu\nu}p, \quad (2.57)$$

where ρ is the energy density of fluid, p is its isotropic pressure, u^μ is the four-velocity of the fluid (normalized so that $g_{\mu\nu}u^\mu u^\nu = 1$), $g_{\mu\nu}$ is the spacetime metric.

Since we are working in the comoving coordinate system where the fluid is at rest, the four-velocity simplifies to,

$$u^\mu = (1, 0, 0, 0), \quad (2.58)$$

$$u_\mu = g_{\mu\nu}u^\nu = (1, 0, 0, 0), \quad (2.59)$$

and the metric $g_{\mu\nu}$ is taken as the Friedmann-Lemaître-Robertson-Walker (FLRW) metric. Using this, we compute components of the stress-energy tensor.

The 00-component is

$$T_{00} = (\rho + p)u_0u_0 - g_{00}p \quad (2.60)$$

$$= \rho. \quad (2.61)$$

The spatial components T_{ij} (for $i = j$) is

$$T_{ij} = (\rho + p)u_iu_j - g_{ij}p \quad (2.62)$$

$$= -g_{ij}p \quad (2.63)$$

$$= a(t)^2\delta_{ij}p, \quad (2.64)$$

which reflects the isotropy and homogeneity of the cosmic fluid. Then substituted into Einstein field equation is

$$G_{\mu\nu} = 8\pi GT_{\mu\nu}, \quad (2.65)$$

where the $\mu = \nu = 0$ component gives the Friedmann equation, which governs the expansion rate of the Universe via the scale factor $a(t)$ we get,

$$\left(\frac{\dot{a}}{a}\right)^2 = \frac{8\pi}{3}G\rho - \frac{\kappa}{a^2}. \quad (2.66)$$

This called Friedmann equation. It connects the rate of expansion of the Universe, characterized by the Hubble parameter $H = \dot{a}/a$ to the total energy density ρ and spatial curvature. However, the Friedmann equation alone is insufficient to fully determine the dynamics, as it involves two time-dependent functions: the scale factor $a(t)$ and the energy density $\rho(t)$. To resolve this, an additional equation is required. A natural choice is to invoke the covariant conservation of the energy-momentum tensor,

$$\nabla_\mu T^{\mu\nu} = \partial_\mu T^{\mu\nu} + \Gamma_{\mu\sigma}^\mu T^{\sigma\nu} + \Gamma_{\mu\sigma}^\nu T^{\mu\sigma} = 0. \quad (2.67)$$

In case $\nu = 0$, we write

$$\nabla_\mu T^{\mu\nu} \equiv \partial_\mu T^{\mu 0} + \Gamma_{\mu\sigma}^\mu T^{\sigma 0} + \Gamma_{\mu\sigma}^0 T^{\mu\sigma} \quad (2.68)$$

$$= 0. \quad (2.69)$$

Non-vanishing components of the energy-momentum tensor in the co-moving frame are

$$T^{00} = g^{00} g^{00} T_{00} \quad (2.70)$$

$$= \rho, \quad (2.71)$$

$$T^{ij} = g^{ik} g^{jl} T_{kl} \quad (2.72)$$

$$= g^{ik} g^{jl} (-g_{kl} p) \quad (2.73)$$

$$= \frac{1}{a^2} \gamma^{ij} p. \quad (2.74)$$

At this point, we utilize the fact that the spatial components of 4-velocity are zero in the co-moving frame, so that $T_{ij} = (p + \rho)u_i u_j - p g_{ij} = -p g_{ij}$. Note that γ^{ij} entering (2.74) is a time-independent component of the inverse matrix of γ_{ij} . We use the expression for the nonzero Christoffel symbol again, and write Eq. (2.68) in the following explicit form,

$$\dot{\rho} + 3\frac{\dot{a}}{a}(\rho + p) = 0. \quad (2.75)$$

This is called continuity equation, an important equation for describing energy conservation. To close the system of equations that defines the dynamics of a

homogeneous and isotropic Universe, it is necessary to specify the equations of state of matter,

$$p = p(\rho). \quad (2.76)$$

The equation presented above does not arise directly from the Einstein field equations of General Relativity. Instead, the equation of state w depends on the physical nature of the matter content in the Universe. For instance, non-relativistic matter yields $p = 0$, relativistic matter corresponds to $p = \frac{1}{3}\rho$, and vacuum energy is characterized by $p = -\rho$. In this analysis, we adopt a spatially flat model with $\kappa = 0$, which serves as a reasonable approximation for our Universe. As will be demonstrated later, the curvature term κ/a^2 in the Friedmann equation is negligible compared to the dominant energy-density term, both in the present epoch and during the early Universe. Under the assumption of spatial flatness, the Friedmann equation simplifies to the following form,

$$H^2 = \frac{\dot{a}^2}{a^2} = \frac{8\pi}{3}G\rho. \quad (2.77)$$

2.4.1 Non-relativistic matter

In this model, the universe is filled with non-relativistic matter whose equation of state is

$$p = 0$$

and

$$\rho = \frac{const}{a^3}$$

There is a simple interpretation: The number density of particles n decreases as the volume of the joint motion increases, and $a^3n = const$, which means conservation of the total number of particles. Since the energy density is given by $\rho = mn$, where m is the mass of the particle, the energy density has the same characteristic, i.e. $\rho a^3 = const$. Considering ρ in this section, Friedmann equation has the following form,

$$\frac{\dot{a}^2}{a^2} = \frac{const}{a^3}, \quad (2.78)$$

and has the solution,

$$a(t) = const \cdot (t - t_s)^{2/3}, \quad (2.79)$$

where t_s is an arbitrary constant, the universe is expanding, and the expansion is decelerating $\ddot{a} < 0$, the energy density is

$$\rho(t) = \frac{const}{(t - t_s)^2}. \quad (2.80)$$

2.4.2 Relativistic matter

In the case where the energy density of the universe is primarily composed of relativistic matter, the equation of state is,

$$p = \frac{1}{3}\rho,$$

and

$$\rho = \frac{const}{a^4}.$$

This behavior differs from non-relativistic matter because the expansion of the universe not only decreases the number density of particles $n \propto a^{-3}$ but also increases the redshift of the energy of each particle $\omega \propto a^{-1}$ (this latter property holds both for non-interacting relativistic particles). We obtain the Friedmann equation is

$$\frac{\dot{a}^2}{a^2} = \frac{const}{a^4}, \quad (2.81)$$

and has solution

$$a(t) = const \cdot t^{1/2}. \quad (2.82)$$

2.4.3 Vacuum

In vacuum (flat space-time), the vacuum is the same in all inertial reference frames, may have a non-zero energy density, and Lorentz stability determines the form of the energy-momentum tensor,

$$T_{\mu\nu} = \rho_{vac}\eta_{\mu\nu}, \quad (2.83)$$

the vacuum energy density is equal to $T_{00} = \rho_{vac}$, while effective pressure, defined by $T_{ij} = -p\eta_{ij}$ satisfies

$$p = -\rho_{vac}.$$

Therefore, vacuum has a novel equation of state $p = -\rho$ for positive energy density, vacuum pressure is negative. In a curved space-time with not very high curvature, the expression $T_{\mu\nu} = \rho_{vac}\eta_{\mu\nu}$ still holds in any local Lorentzian frame and in an arbitrary frame, one would have

$$T_{\mu\nu} = \rho_{vac}g_{\mu\nu}, \quad (2.84)$$

where the vacuum energy density, ρ_{vac} is spatially and temporally constant. In principle, a complete theory of elementary particles and their interactions should be able to predict its value. However, no convincing calculation has yet been achieved, making this one of the central unresolved issues in fundamental physics. A time-independent ρ_{vac} is consistent with eq.2.75, which, for $p = -\rho$, leads to $\dot{\rho} = 0$. This result follows naturally from the covariant conservation of the energy–momentum tensor. Specifically, the tensor given in eq.2.84 satisfies the conservation law for $\rho_{vac} = const$, owing to the identity $\nabla_\mu g^{\lambda\rho} = 0$.

From another perspective, the right-hand side of the Einstein field equations with $T_{\mu\nu} = const \cdot g_{\mu\nu}$ can be interpreted through the principle of general covariance, which permits the addition of a term to the Einstein–Hilbert action S_{EH} of General Relativity,

$$S_{EH} = -\frac{1}{16\pi G} \int d^4x \sqrt{-g} R, \quad (2.85)$$

and

$$S_\Lambda = -\Lambda \int \sqrt{-g} d^4x, \quad (2.86)$$

where S_Λ is Cosmological constant term, by the varying action $S_{EH} + S_\Lambda$,

$$\delta S = \delta S_{EH} + \delta S_\Lambda. \quad (2.87)$$

In case of δS_Λ , we can vary action as follows,

$$\delta S_\Lambda = -\Lambda \int d^4x \delta(\sqrt{-g}), \quad (2.88)$$

from this identity of the variation of the determinant,

$$\delta \det M = (\det M) \text{Tr}(M^{-1} \delta M), \quad (2.89)$$

then, we apply this relation (2.89) which will get

$$\delta\sqrt{-g} = \frac{1}{2}\sqrt{-g}g^{\mu\nu}\delta g_{\mu\nu}, \quad (2.90)$$

this given δS_Λ as follows

$$\delta S_\Lambda = -\frac{\Lambda}{2} \int d^4x \sqrt{-g} g^{\mu\nu} \delta g_{\mu\nu}. \quad (2.91)$$

Next, we will vary action of δS_{EH} in the following steps

$$\delta S_{EH} = -\frac{1}{16\pi G} \int d^4x \delta(\sqrt{-g}R) \quad (2.92)$$

$$= -\frac{1}{16\pi G} \int d^4x (\delta\sqrt{-g}R + \sqrt{-g}\delta R), \quad (2.93)$$

where

$$\delta R = R_{\mu\nu}\delta g^{\mu\nu} + g^{\mu\nu}\delta R_{\mu\nu}, \quad (2.94)$$

so we obtain,

$$\delta S_{EH} = -\frac{1}{16\pi G} \int d^4x (\delta\sqrt{-g}R + \sqrt{-g}R_{\mu\nu}\delta g^{\mu\nu} + \sqrt{-g}g^{\mu\nu}\delta R_{\mu\nu}). \quad (2.95)$$

We will divide δS_{EH} into three parts as follows,

$$\delta S_{EH} = \delta S_{EH1} + \delta S_{EH2} + \delta S_{EH3}, \quad (2.96)$$

where

$$\delta S_{EH1} = -\frac{1}{16\pi G} \int d^4x (\delta\sqrt{-g}R), \quad (2.97)$$

$$\delta S_{EH2} = -\frac{1}{16\pi G} \int d^4x (\sqrt{-g}R_{\mu\nu}\delta g^{\mu\nu}), \quad (2.98)$$

and

$$\delta S_{EH3} = -\frac{1}{16\pi G} \int d^4x (\sqrt{-g}g^{\mu\nu}\delta R_{\mu\nu}). \quad (2.99)$$

Next, we will calculate each part, starting with the first part δS_{EH1} ,

$$\delta S_{EH1} = -\frac{1}{16\pi G} \int d^4x (\delta\sqrt{-g}R) \quad (2.100)$$

$$= -\frac{1}{32\pi G} \int d^4x \sqrt{-g} R g^{\mu\nu} \delta g_{\mu\nu}, \quad (2.101)$$

then we do the calculation of δS_{EH2} ,

$$\delta S_{EH2} = -\frac{1}{16\pi G} \int d^4x (\sqrt{-g} R_{\mu\nu} \delta g^{\mu\nu}), \quad (2.102)$$

from identity of matrix,

$$g_{\rho\lambda} \delta g^{\mu\rho} = -g^{\mu\rho} \delta g_{\rho\lambda}, \quad (2.103)$$

contracting both sides of the matrix with $g^{\lambda\nu}$ we obtain the relation

$$\delta g^{\mu\nu} = -g^{\mu\rho} \delta g_{\rho\lambda} g^{\lambda\nu}, \quad (2.104)$$

therefore,

$$\delta S_{EH2} = \frac{1}{16\pi G} \int d^4x (\sqrt{-g} R^{\mu\nu} \delta g_{\mu\nu}). \quad (2.105)$$

The final step is to evaluate the variation δS_{EH3} , which at first sight appears to be the most involved term. To carry this out, we recall that the transformation properties of the Christoffel symbols ensure that $\delta \Gamma_{\nu\lambda}^{\mu}$ behaves as a tensor. Applying relation Riemann tensor,

$$R_{\nu\lambda\rho}^{\mu} = \partial_{\lambda} \Gamma_{\nu\rho}^{\mu} - \partial_{\rho} \Gamma_{\nu\lambda}^{\mu} + \Gamma_{\sigma\lambda}^{\mu} \Gamma_{\nu\rho}^{\sigma} - \Gamma_{\sigma\rho}^{\mu} \Gamma_{\nu\lambda}^{\sigma}, \quad (2.106)$$

we then obtain the following expression for the variation of the Riemann tensor,

$$\delta R_{\nu\lambda\rho}^{\mu} = \partial_{\lambda} \delta \Gamma_{\nu\rho}^{\mu} - \partial_{\rho} \delta \Gamma_{\nu\lambda}^{\mu} + \delta \Gamma_{\sigma\lambda}^{\mu} \Gamma_{\nu\rho}^{\sigma} + \Gamma_{\sigma\lambda}^{\mu} \delta \Gamma_{\nu\rho}^{\sigma} - \delta \Gamma_{\sigma\rho}^{\mu} \Gamma_{\nu\lambda}^{\sigma} - \Gamma_{\sigma\rho}^{\mu} \delta \Gamma_{\nu\lambda}^{\sigma}. \quad (2.107)$$

By direct calculation it can be verified that,

$$\delta R_{\nu\lambda\rho}^{\mu} = \nabla_{\lambda} (\delta \Gamma_{\nu\rho}^{\mu}) - \nabla_{\rho} (\delta \Gamma_{\nu\lambda}^{\mu}), \quad (2.108)$$

where the covariant derivatives are defined with respect to the background (unperturbed) metric. From previous relation, we obtain the following expression for the variation of the Ricci tensor,

$$\delta R_{\mu\nu} = \nabla_{\lambda} (\delta \Gamma_{\mu\nu}^{\lambda}) - \nabla_{\nu} (\delta \Gamma_{\mu\lambda}^{\lambda}). \quad (2.109)$$

Then substituting in δS_{EH3} , we get

$$\delta S_{EH3} = -\frac{1}{16\pi G} \int d^4x (\sqrt{-g} g^{\mu\nu} \delta R_{\mu\nu}) \quad (2.110)$$

$$= -\frac{1}{16\pi G} \int d^4x (\sqrt{-g} g^{\mu\nu} [\nabla_{\lambda} (\delta \Gamma_{\mu\nu}^{\lambda}) - \nabla_{\nu} (\delta \Gamma_{\mu\lambda}^{\lambda})]) \quad (2.111)$$

$$= -\frac{1}{16\pi G} \int d^4x (\sqrt{-g} \nabla_{\lambda} (g^{\mu\nu} \delta \Gamma_{\mu\nu}^{\lambda}) - g^{\mu\lambda} \delta \Gamma_{\mu\sigma}^{\sigma}). \quad (2.112)$$

In the second equality, the metric tensor $g_{\mu\nu}$ has been placed inside the covariant derivatives, and the summation indices ν and λ have been relabeled in the second term. Utilizing the property,

$$\nabla_\mu A^\mu = \frac{1}{\sqrt{-g}} \partial_\mu (\sqrt{-g} A^\mu), \quad (2.113)$$

we can finally express δS_{EH3} as an integral of a total derivative:

$$\delta S_{EH3} = -\frac{1}{16\pi G} \int d^4x \partial_\lambda [\sqrt{-g} (g^{\mu\nu} \delta \Gamma_{\mu\nu}^\lambda - g^{\mu\lambda} \delta \Gamma_{\mu\sigma}^\sigma)]. \quad (2.114)$$

Which is the integral divided by the total derivative, meaning that the result is a boundary term. However, we assume that for realistic fields these terms vanish as we approach infinity, so this term will be zero, so when we combine δS_{EH1} and δS_{EH2} , we get the Einstein–Hilbert variation action,

$$\delta S_{EH} = \frac{1}{16\pi G} \int d^4x \sqrt{-g} \left(R^{\mu\nu} - \frac{1}{2} g^{\mu\nu} R \right) \delta g_{\mu\nu}. \quad (2.115)$$

From varying the action ($\delta S_{EH} + \delta S_\Lambda$) with respect to the metric, the following equations are obtained (without matter),

$$R_{\mu\nu} - \frac{1}{2} g_{\mu\nu} R = 8\pi G \Lambda g_{\mu\nu}. \quad (2.116)$$

This is an equation called Einstein field equations in vacuum (2.116) used to describe the dynamics of gravitation field in vacuum. However, the first important thing is to study the gravity in the field of matter which acts as the source of the gravitational field. To study this general situation, we will add the action of matter in a new term,

$$S_m = \int d^4x \sqrt{-g} \mathcal{L}_m \quad (2.117)$$

and the varying action of matter is defined by the following this relation,

$$\delta S_m = -\frac{1}{2} \int d^4x \sqrt{-g} T^{\mu\nu} \delta g_{\mu\nu}, \quad (2.118)$$

where $T_{\mu\nu}$ is energy-momentum tensor and is defined by

$$T_{\mu\nu} = -\frac{2}{\sqrt{-g}} \frac{\delta S_m}{\delta g^{\mu\nu}}. \quad (2.119)$$

From all of this we get the Einstein field equation with matter in this formula,

$$R_{\mu\nu} - \frac{1}{2} g_{\mu\nu} R = 8\pi G (\Lambda g_{\mu\nu} + T). \quad (2.120)$$

From the Einstein field equation, it is clear that the geometry of spacetime is deeply linked to the distribution of matter and energy, represented by the energy-momentum tensor $T_{\mu\nu}$. To solve these equations for any physical system, a specific mathematical form for $T_{\mu\nu}$ is required. This is achieved by defining the equations of state (EoS), which are thermodynamic relations that describe the macroscopic properties of matter sources. EoS typically relates the pressure (p) of a matter to its energy density (ρ). Therefore, a detailed analysis of the EoS for different cosmological components is an important next step in understanding the dynamics of the Universe.

In case of vacuum energy and the cosmological constant are equivalent in this context. General barotropic equation of state ($p = w\rho$), this topics are generalizes the previous model (dust, radiation, and vacuum) to include any barotropic fluid. The pressure is a constant fraction of the energy density,

$$p = w\rho,$$

where w is a equation of state parameter

$$w = \begin{cases} -1 & \text{vacuum energy (cosmological constant),} \\ 0 & \text{non-relativistic matter (dust),} \\ 1/3 & \text{relativistic matter (radiation).} \end{cases} \quad (2.121)$$

If w is a negative models ($w < -1/3$) are relevant for dark energy and quintessence models. The energy density scaling from the continuity equation (2.75) can be write as,

$$\dot{\rho} + 3\frac{\dot{a}}{a}(\rho + p) = 0 \Rightarrow \rho \propto a^{-3(1+w)}. \quad (2.122)$$

The energy density decreases depending on w (faster for higher w). For the scale factor evolution, after solving the Friedmann equation for the flat universe,

$$\frac{\dot{a}^2}{a^2} = \frac{8\pi}{3}G\rho \Rightarrow a(t) \propto t^\alpha, \quad (2.123)$$

where

$$\alpha = \frac{2}{3(1+w)}. \quad (2.124)$$

The expansion of the Universe are acceleration or deceleration depend on

$$\ddot{a} = \alpha(\alpha - 1)t^{\alpha-2}, \quad (2.125)$$

we can determine the expansion behavior,

$$\alpha \begin{cases} > 1 (w < -1/3) & \text{acceleration,} \\ < 1 (w > -1/3) & \text{deceleration,} \\ = 1 (w = -1/3) & \text{coasting (neither acceleration nor deceleration).} \end{cases} \quad (2.126)$$

2.5 Λ CDM: Cosmological Model with Dark Energy and Dark Matter

In this topic present about the Λ CDM model, the Lambda-Cold Dark Matter (Λ CDM) model represents the standard model of modern cosmology. It provides a remarkably successful framework for describing the composition, evolution, and large-scale structure of the universe. This model is built upon the foundation of Einstein's General Relativity and posits that the universe is composed of several key ingredients. According to the latest observational data, the energy budget of the present-day universe is dominated by two mysterious components: dark energy (Λ) and cold dark matter (CDM). The composition of the universe can be summarized by the density parameters Ω_i ,

$$\Omega_i = \rho_i / \rho_c \quad (2.127)$$

where $\rho_c = 3H_0^2/8\pi G$ is the critical density required for a spatially flat universe.

The primary components of the Universe can be categorized as dark energy, matter, and radiation.

Dark energy (Ω_Λ) is an unknown form of energy that permeates all of space and is responsible for the accelerated expansion of the universe, effectively acting as a repulsive force that pushes space to expand. At this time, the estimated value of dark energy,

$$\Omega_\Lambda \approx 0.685.$$

Matter (Ω_M) accounts for all components that contribute to gravitational attraction. It is generally divided into two parts, Cold Dark Matter (Ω_{DM}) and

Baryonic Matter (Ω_B). Cold dark matter is a non-baryonic, non-relativistic form of matter that does not interact with light but provides the gravitational scaffolding for the formation of cosmic structures. In contrast, baryonic matter corresponds to the ordinary matter that constitutes stars, planets, dust, meteorites, and interstellar gas. The total matter density is given by

$$\Omega_M = \Omega_{DM} + \Omega_B,$$

with current estimates

$$\Omega_{DM} \approx 0.265, \quad \Omega_B \approx 0.050.$$

Finally, radiation (Ω_{rad}) consists of photons and neutrinos. This component dominated the energy density of the Universe in its earliest stages but has become negligible in the present epoch.

Nowadays the observational evidence, particularly from the Cosmic Microwave Background (CMB), strongly suggests that the universe is spatially flat, meaning the total density parameter,

$$\Omega_{total} = \Omega_\Lambda + \Omega_M + \Omega_{rad} \approx 1.$$

For an expanding, homogeneous and isotropic universe there will be dynamics are governed by the Friedmann equation, derived from Einstein field equations. In the context of the Λ CDM model, this equation relates the Hubble parameter H , to the energy densities of the universe's components,

$$H^2 = \left(\frac{\dot{a}}{a}\right)^2 = \frac{8\pi G}{3}(\rho_M + \rho_{rad} + \rho_\Lambda), \quad (2.128)$$

where $a(t)$ is scale factor. By expressing the energy densities in term of their present days values and their dependence on scale factor (a) or redshift (z), we can write the relation in this form,

$$H^2 = H_0^2[\Omega_M(1+z)^3 + \Omega_{rad}(1+z)^4 + \Omega_\Lambda], \quad (2.129)$$

or

$$H^2 = H_0^2 \left[\Omega_M \left(\frac{1}{a}\right)^3 + \Omega_{rad} \left(\frac{1}{a}\right)^4 + \Omega_\Lambda \right]. \quad (2.130)$$

Each term describes the contribution from matter, radiation, dark energy (cosmological constant). When we observe the different powers of $(1 + z)$ or a reflect the distinct physical behavior of each components. Matter ($\rho \propto a^{-3}$) are the density of non-relativistic matter (both dark matter and baryonic) decreases as the volume of space increases. Radiation ($\rho_{rad} \propto a^{-4}$) are the density of radiation decreases not only due to increasing volume but also because its energy is redshifted as the universe expanded. Dark Energy ($\rho_\Lambda \propto a^0$), in the simplest model where dark energy is a cosmological constant (Λ), its energy density remains constant throughout cosmic history. This equation forms the backbone of the Λ CDM model, allowing us to trace the expansion history of the universe from its early moments to the present day.

The role of Λ CDM and evolution of cosmological, in the Λ CDM model are predicts a specific sequence of epochs, each dominated by a different component.

- Radiation-Dominated Era: In the very early universe ($z \gg 3400$), the energy density was dominated by radiation. The universe expanded but this expansion was decelerating due to the strong self-gravity of radiation.
- Matter-Dominated Era: As the universe expanded and cooled, the energy density of matter, which dilutes slower than radiation, eventually became dominant. This transition, known as matter-radiation equality, occurred at a redshift of $z_{eq} \approx 3400$. During this era, the expansion continued to decelerate, allowing for the gravitational collapse of matter overdensities, which seeded the formation of galaxies and large-scale structures. The presence of Cold Dark Matter was crucial during this period; being immune to radiation pressure, it could begin clumping together much earlier than baryonic matter, creating the gravitational potential wells into which baryons later fell.
- Dark Energy-Dominated Era: In the recent cosmological past, at a redshift of $z \approx 0.3$, the constant energy density of dark energy finally surpassed the diluting density of matter. This marked a profound shift in cosmic dynamics. Since dark energy, modeled as a cosmological constant, has a negative pressure ($p_\Lambda = -\rho_\Lambda$), it exerts a repulsive gravitational force. Consequently,

the expansion of the universe transitioned from deceleration to acceleration, a key prediction of the Λ CDM model confirmed by observations of Type Ia supernovae.

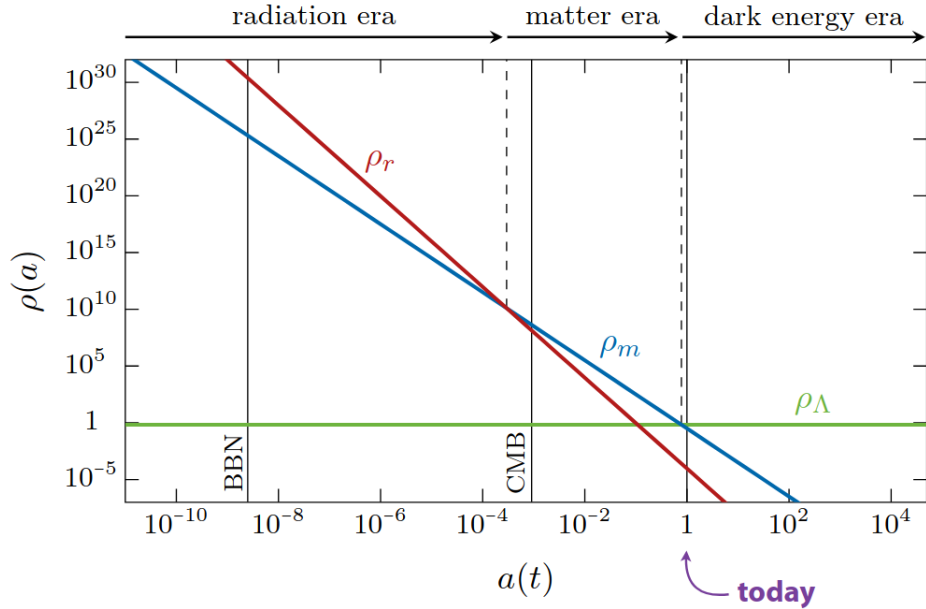


Figure 2.2 Evolution of the Universe across different cosmological epochs. At early times, the expansion is dominated by radiation, followed by a matter-dominated era in which large-scale structure formation occurs. At late times, the accelerated expansion is driven by dark energy, shaping the present-day universe [22].

While the Λ CDM model is phenomenologically successful, the fundamental nature of its two main components remains one of the greatest unsolved mysteries in physics. Cold Dark Matter (CDM): The "Cold" in CDM signifies that these particles were moving at non-relativistic speeds during the era of structure formation. This property is essential for the "bottom-up" formation of cosmic structures (small structures form first and merge to create larger ones). Leading candidates for CDM particles are hypothetical, weakly interacting massive particles (WIMPs) or axions.

Dark Energy (Λ): The simplest explanation for dark energy is the cosmological constant (Λ), representing the energy of the vacuum itself. While this fits the observational data well (with an equation of state parameter $w = p/\rho = -1$),

theoretical calculations of vacuum energy are famously discrepant from the observed value by many orders of magnitude. Alternative models, such as quintessence, propose that dark energy is a dynamic scalar field with a slowly changing energy density.

In conclusion, the Λ CDM model stands as a robust and elegant description of our universe, successfully explaining a vast range of cosmological observations from the early universe's afterglow to the accelerated expansion of the present day. However, its reliance on two unknown entities, dark matter and dark energy, highlights the frontier of modern cosmology and particle physics.

2.6 Evolution of scalar fields

Scalar fields play a central role in many areas of modern cosmology. They are employed in theoretical frameworks that describe the early universe, such as inflation, and are also proposed as candidates for dark energy and dark matter. In particular, scalar fields with suitable potentials can exhibit a wide range of cosmological behaviors depending on their initial conditions and the evolution of the background spacetime. A prominent example of such a scalar field is the axion, whose dynamics are governed by the Klein–Gordon equation in an expanding universe.

To study the evolution of a scalar field, let us consider a spatially flat Friedmann–Lemaître–Robertson–Walker (FLRW) metric of the form,

$$ds^2 = dt^2 - a^2(t)d\vec{x}^2, \quad (2.131)$$

where $a(t)$ is the scale factor. Assuming a spatially homogeneous scalar field $\phi(t)$, the action governing its dynamics is given by

$$S = \int d^4x \sqrt{-g} \left[\frac{1}{2} g^{\mu\nu} \partial_\mu \phi \partial_\nu \phi - V(\phi) \right], \quad (2.132)$$

where $V(\phi)$ is the scalar field potential. We consider a flat FLRW universe containing a single homogeneous scalar field $\phi(t)$ that is minimally coupled to gravity. By varying action, the evolution of this system is governed by the Klein–Gordon and Friedmann equations: We vary the field as $\phi \rightarrow \phi + \delta\phi$. The variation of the action

is

$$\delta S = \int d^4x \sqrt{-g} (g^{\mu\nu} \partial_\mu \phi \partial_\nu (\delta\phi) - V'(\phi) \delta\phi). \quad (2.133)$$

Using the identity

$$\sqrt{-g} g^{\mu\nu} \partial_\mu \phi \partial_\nu (\delta\phi) = \partial_\nu (\sqrt{-g} g^{\mu\nu} \partial_\mu \phi \delta\phi) - \partial_\nu (\sqrt{-g} g^{\mu\nu} \partial_\mu \phi) \delta\phi, \quad (2.134)$$

we obtain

$$\delta S = - \int d^4x \delta\phi [\partial_\nu (\sqrt{-g} g^{\mu\nu} \partial_\mu \phi) + \sqrt{-g} V'(\phi)], \quad (2.135)$$

where the total divergence term vanishes due to $\delta\phi = 0$ at the boundary. Requiring $\delta S = 0$ for arbitrary $\delta\phi$ gives

$$\frac{1}{\sqrt{-g}} \partial_\nu (\sqrt{-g} g^{\mu\nu} \partial_\mu \phi) + V'(\phi) = 0. \quad (2.136)$$

Defining the covariant d'Alembertian

$$\square_g \phi \equiv \nabla^\mu \nabla_\mu \phi = \frac{1}{\sqrt{-g}} \partial_\mu (\sqrt{-g} g^{\mu\nu} \partial_\nu \phi), \quad (2.137)$$

the equation of motion becomes

$$\square_g \phi + V'(\phi) = 0, \quad (2.138)$$

which is the Klein–Gordon equation in curved spacetime. For a quadratic potential $V(\phi) = \frac{1}{2}m^2\phi^2$, this reduces to

$$\square_g \phi + m^2 \phi = 0. \quad (2.139)$$

For the flat FLRW metric in cosmic time

$$ds^2 = -dt^2 + a(t)^2 d\mathbf{x}^2, \quad \sqrt{-g} = a^3, \quad (2.140)$$

the operator becomes

$$\square_g \phi = -\frac{1}{a^3} \partial_t (a^3 \dot{\phi}) + \frac{1}{a^2} \nabla^2 \phi. \quad (2.141)$$

Hence the Klein–Gordon equation is

$$\ddot{\phi} + 3H\dot{\phi} - \frac{1}{a^2} \nabla^2 \phi + V'(\phi) = 0, \quad (2.142)$$

with $H = \dot{a}/a$ the Hubble parameter. For a spatially homogeneous field $\phi = \phi(t)$, the Laplacian term vanishes, giving

$$\ddot{\phi} + 3H\dot{\phi} + V'(\phi) = 0, \quad (2.143)$$

which is the standard form used in cosmology. For a massive scalar, $V'(\phi) = m^2\phi$, this becomes

$$\ddot{\phi} + 3H\dot{\phi} + m^2\phi = 0. \quad (2.144)$$

The term $3H\dot{\phi}$ acts as a friction term due to the expansion of spacetime, and this equation is structurally analogous to a damped oscillator.

The evolution of the scalar field can be classified into two dynamical regimes depending on the relative importance of the friction term and the potential gradient. In the slow-roll regime, where $|\ddot{\phi}| \ll |3H\dot{\phi}|$ and $|\dot{\phi}^2| \ll |V(\phi)|$, the field evolves slowly and its dynamics are dominated by the potential. This behavior is typical of inflationary or quintessence models, in which the scalar field effectively behaves like a cosmological constant with an equation of state $w \approx -1$. In contrast, in the oscillatory regime, when the Hubble friction becomes negligible compared to the curvature of the potential, the scalar field oscillates around the minimum of its potential. If the potential is approximately quadratic in that region, the time-averaged equation of state approaches $w \approx 0$, causing the field to mimic pressureless matter.

The energy density and pressure associated with a homogeneous scalar field are given by

$$\rho_\phi = \frac{1}{2}\dot{\phi}^2 + V(\phi), \quad (2.145)$$

$$p_\phi = \frac{1}{2}\dot{\phi}^2 - V(\phi), \quad (2.146)$$

which yield an effective equation of state parameter

$$w = \frac{p_\phi}{\rho_\phi}. \quad (2.147)$$

In the context of axion cosmology, the scalar field typically exhibits a periodic potential of the form

$$V(\phi) = m^2 f_a^2 \left[1 - \cos \left(\frac{\phi}{f_a} \right) \right], \quad (2.148)$$

where m is the axion mass and f_a is the axion decay constant. Near the minimum, this potential can be approximated by a quadratic form,

$$V(\phi) \approx \frac{1}{2}m^2\phi^2. \quad (2.149)$$

In this regime, the scalar field evolves as a damped harmonic oscillator. The onset of oscillations occurs when the Hubble parameter $H(t)$ drops below the axion mass m , signaling a transition from an overdamped regime to an underdamped one. This behavior underpins the so-called misalignment mechanism, which governs the relic abundance of axionic dark matter.

During oscillations, the energy density of the scalar field scales as $\rho_\phi \propto a^{-3}$, analogous to non-relativistic matter. This scaling reinforces the interpretation of oscillating scalar fields as effective cold dark matter components. The capability of scalar fields to interpolate between vacuum energy-like behavior and matter-like behavior underlines their importance in both early and late-time cosmological dynamics.

In summary, the evolution of scalar fields in an expanding universe exhibits rich dynamical behavior controlled by the form of the potential and the expansion rate of the universe. These properties make scalar fields indispensable tools in modeling various cosmological phenomena, from inflation to dark energy and dark matter. In particular, axion fields provide a compelling and well-motivated example where such dynamics play a crucial role in connecting particle physics to cosmology.

2.6.1 Axion cosmology

The dynamics of scalar fields are fundamental to modern cosmology, shaping phenomena that range from inflation to dark matter models involving axion-like particles. Axions, in particular, are pseudo-scalar fields that arise both as a solution to the strong CP problem in quantum chromodynamics (QCD) and as generic predictions in string theory and other high-energy frameworks. Cosmologically, axions evolve as coherent classical fields in an expanding Friedmann–Lemaître–Robertson–Walker (FLRW) universe, with their behavior governed by the Klein–Gordon equation. At early times, Hubble friction prevents

the field from evolving, leaving it nearly constant. Once the Hubble expansion rate drops below the axion mass scale ($H \sim m_a$), the field begins oscillating about the minimum of its potential. These oscillations behave like non-relativistic matter, with the associated energy density diluting as $\rho_a \propto a^{-3}$, thereby establishing axions as compelling cold dark matter candidates. Through this mechanism, axions provide a natural link between fundamental particle physics and cosmological observables, influencing processes from inflationary dynamics to structure formation and the anisotropies of the cosmic microwave background [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12].

In this work, we consider a flat Friedmann–Lemaître–Robertson–Walker (FLRW) universe containing a single homogeneous scalar field $\phi(t)$ that is minimally coupled to gravity. The evolution of this system is governed by the Klein–Gordon and Friedmann equations,

$$\ddot{\phi} + 3H\dot{\phi} + m_a^2\phi = 0, \quad (2.150)$$

$$H^2 = \frac{1}{3}(\rho_\phi + \rho_m + \rho_r + \rho_\Lambda), \quad (2.151)$$

where $H = \dot{a}/a$ is the Hubble parameter, and the scalar field energy density is given by $\rho_\phi = \frac{1}{2}\dot{\phi}^2 + \frac{1}{2}m_a^2\phi^2$. The quantities ρ_m , ρ_r , and ρ_Λ represent the energy densities of matter, radiation, and the cosmological constant, respectively.

To study the dynamical behavior of the field, we found that axions can exhibit dramatically different cosmological behavior depending on their mass scale. For small mass axions (e.g., $m_a \leq 10^{-30}$ eV), the field remains overdamped due to the Hubble expansion throughout much of the cosmic history, preventing it from oscillating. As a result, the axion field evolves slowly and behaves like a nearly constant energy component with an equation of state $w \approx -1$, mimicking dark energy. This regime is sometimes referred to as “axion dark energy.” On the other hand, large mass axions (e.g., $m_a \geq 10^{-25}$ eV) begin oscillating early—typically before matter-radiation equality—when the Hubble parameter drops below their mass $H \sim m_a$. In this regime, the axion field undergoes coherent oscillations around the minimum of its potential, and its averaged energy density redshifts as $\rho_a \propto a^{-3}$, making it an excellent candidate for cold dark matter. Between these two regimes

lies an intermediate “constrained” region, where the axion begins oscillation too late to cluster efficiently and too early to behave as dark energy, leading to observable suppression in the matter power spectrum. This intermediate range of axion masses is strongly constrained by cosmological observations [5, 23], this will be the axion mass range that we will study.

In this chapter, we introduce the background of cosmology. In the next chapter, we are going to present the neural network.

CHAPTER III

NEURAL NETWORKS

Neural Networks or Artificial Neural Networks (ANNs) are a class of computational models inspired by the structure and functioning of biological neural systems. They are designed to approximate complex nonlinear mappings between inputs and outputs by composing simple computational units, referred to as neurons. Each neuron processes a weighted sum of its inputs, applies a nonlinear or linear activation function, and propagates the result to the subsequent layers. Through the composition of many such units, neural networks are capable of representing highly complex functions and learning patterns from large-scale data.

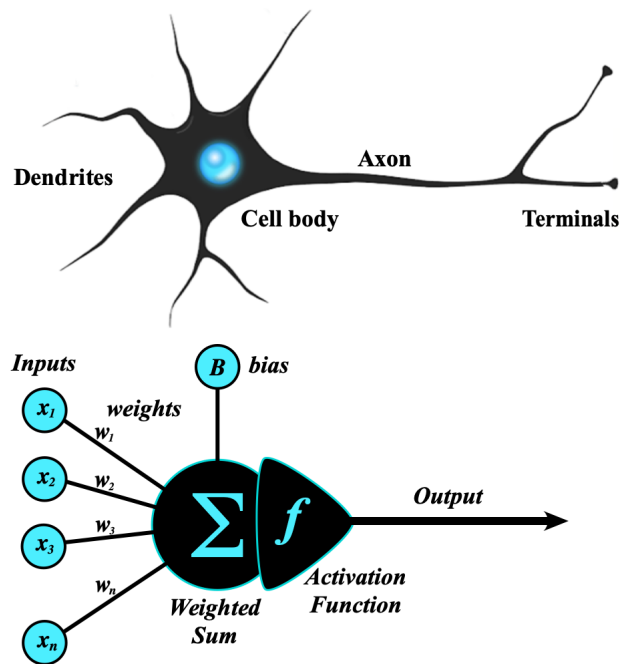


Figure 3.1 Similarities and differences between biologic and artificial neurons [24].

The historical development of neural networks can be traced back to the perceptron model introduced by Rosenblatt in 1958 [25], which provided a simplified abstraction of a biological neuron. Although early enthusiasm declined due to limitations highlighted by Minsky and Papert in the late 1960s [26], the

resurgence of interest occurred with the introduction of the backpropagation algorithm. Backpropagation enabled efficient training of multilayer architectures by propagating error gradients backward through the network, thus overcoming the restrictions of the single-layer perceptron.

In the modern era, neural networks have become central to the field of machine learning, owing to advancements in computational power, optimization techniques, and the availability of large datasets. Deep Neural Networks (DNNs) or Deep learning, which involve architectures with multiple hidden layers, have demonstrated remarkable success across diverse domains such as computer vision, natural language processing, reinforcement learning, and scientific modeling. Their ability to extract hierarchical representations of data makes them especially powerful in solving problems with high-dimensional and nonlinear structures.

Within the scientific community, neural networks are increasingly utilized as surrogate models, solvers for partial differential equations, and tools for data-driven discovery. In particular, Physics-Informed Neural Networks (PINNs) have recently emerged as a novel approach to integrate physical laws, expressed as differential equations, directly into the training objective of neural networks. This hybrid methodology bridges the gap between purely data-driven learning and physics-based modeling, providing a flexible framework for addressing complex problems in areas such as fluid dynamics, quantum mechanics, and cosmology.

Multilayer Perceptrons (MLPs), the simplest feedforward Neural Networks (NNs), are provable universal function approximator. As such, they have been used as a foundation to design complex models, which have drawn significant attention over the past decade, capable of solving difficult data-driven problems, from image classification to speech-to-text task. Here, we briefly describe the core components of neural networks.

In a feed-forward neural network, each layer consists of a number of units (neurons), that first computes a weighted linear combination,

$$\mathbf{z} = \mathbf{W}^T \mathbf{x} + \mathbf{b}, \quad (3.1)$$

where $\mathbf{z}$ is the output of the node in the layer, $\mathbf{W}_i$ the weight of the model parameters,

the input of the layer x and $\mathbf{b}$ are bias terms that need to be estimated using data-driven training.

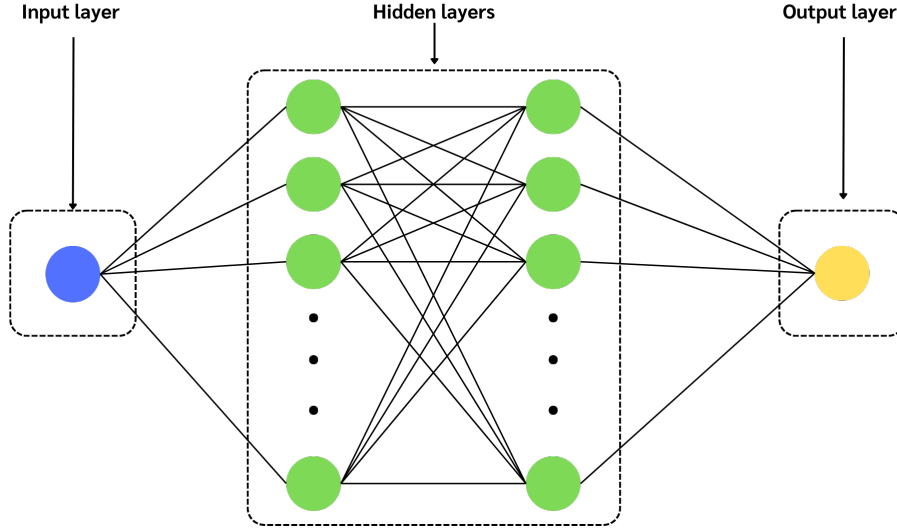


Figure 3.2 Structure of neural network with three hidden layers and shows the basic structure of a multilayer perceptron. Technically, a multilayer perceptron is a special kind of neural network

In addition, the choice of a nonlinear activation function σ must be specified. For many of the fully connected internal layers, this non-linearity is typically a smooth squashing function, such as $\tanh$. By successively applying such transformations, potentially with different non-linear functions and varying numbers of neuron between layers, the network produces an output $\mathbf{z}$. Architectures constructed in this manner with multiple layers are referred to as Deep Neural Networks (DNNs). To enhance their representational capacity and capture more complex latent structures, modern Deep Neural Networks (DNNs) incorporate additional components such as pooling (or other compressive) layers, batch normalization techniques, and activation functions like the Rectified Linear Unit (ReLU).

In general, neural networks function under the assumption that transformations between layers can be sequentially composed, with each layer representing a (nonlinear) function of the preceding one. For the first hidden layer z , given the input x , this can be expressed as:

$$\mathbf{z} = \sigma(\mathbf{W}^T x + \mathbf{b}), \quad (3.2)$$

and for n layers

$$\mathbf{z}_n = \sigma(\mathbf{W}_n^T \mathbf{z}_{n-1} + \mathbf{b}_n). \quad (3.3)$$

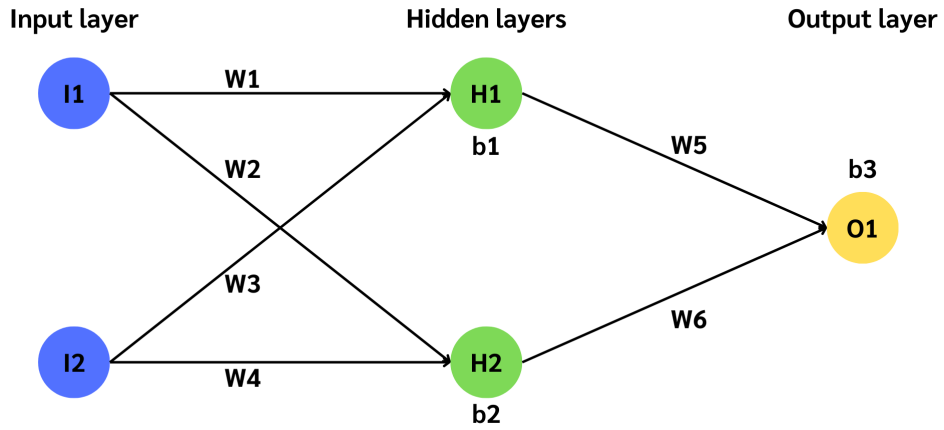


Figure 3.3 Example of neural network calculation.

Here is an example of the calculation for each layer, referring to the following figure, which shows that,

$$\mathbf{H} = \sigma(\mathbf{W}^T x + \mathbf{b}) = \sigma \left(\begin{bmatrix} \mathbf{W1} & \mathbf{W3} \\ \mathbf{W2} & \mathbf{W4} \end{bmatrix} \begin{bmatrix} \mathbf{I1} \\ \mathbf{I2} \end{bmatrix} + \begin{bmatrix} \mathbf{b1} \\ \mathbf{b2} \end{bmatrix} \right), \quad (3.4)$$

and

$$\mathbf{O} = \sigma(\mathbf{W}^T x + \mathbf{b}) = \sigma \left(\begin{bmatrix} \mathbf{W5} & \mathbf{W6} \end{bmatrix} \begin{bmatrix} \mathbf{H1} \\ \mathbf{H2} \end{bmatrix} + \mathbf{b3} \right). \quad (3.5)$$

Following the linear transformation $\mathbf{z} = \sigma(\mathbf{W}^T x + \mathbf{b})$, each neuron applies a nonlinear activation function σ to its output. This step is of paramount importance; it is the primary source of the network's ability to model complex, non-linear relationships in data. Without a non-linear activation function, a multi-layer neural network, no matter how deep, would simply be a linear model. The composition of

multiple linear transformations is itself just a single linear transformation. Therefore, the introduction of non-linearity is what endows neural networks with the power of universal function approximation, allowing them to learn intricate patterns that are not linearly separable. The choice of activation function significantly impacts the network's performance and training dynamics. Below are some of the most commonly used activation functions in modern neural networks. The following will present an example of activation.

Sigmoid (Logistic): The sigmoid function was historically popular, especially in the output layer of binary classification models. It squashes its input value into a range between 0 and 1.

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (3.6)$$

But there is often the problem of the vanishing gradient. For very large or very small input values, the function's gradient approaches zero, which can stall the learning process during backpropagation. Its output is also not zero-centered, which can slow down convergence.

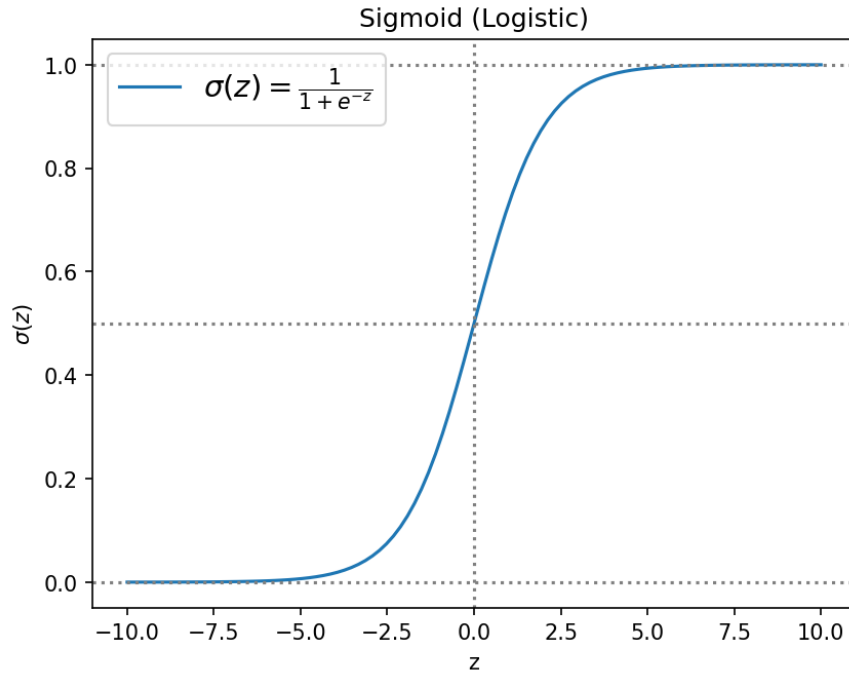


Figure 3.4 The Sigmoid activation function, mapping inputs smoothly to the range $(0, 1)$.

Hyperbolic Tangent (tanh): The hyperbolic tangent function is mathematically similar to the sigmoid but squashes its input to a range between -1 and 1,

$$\sigma(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}. \quad (3.7)$$

This activation function has advantage of being zero-centered, which makes it generally preferable to the sigmoid function for hidden layers, as it can help speed up the convergence of optimization algorithms like gradient descent. The drawback is that it still suffers from the vanishing gradient problem in its saturated regions.

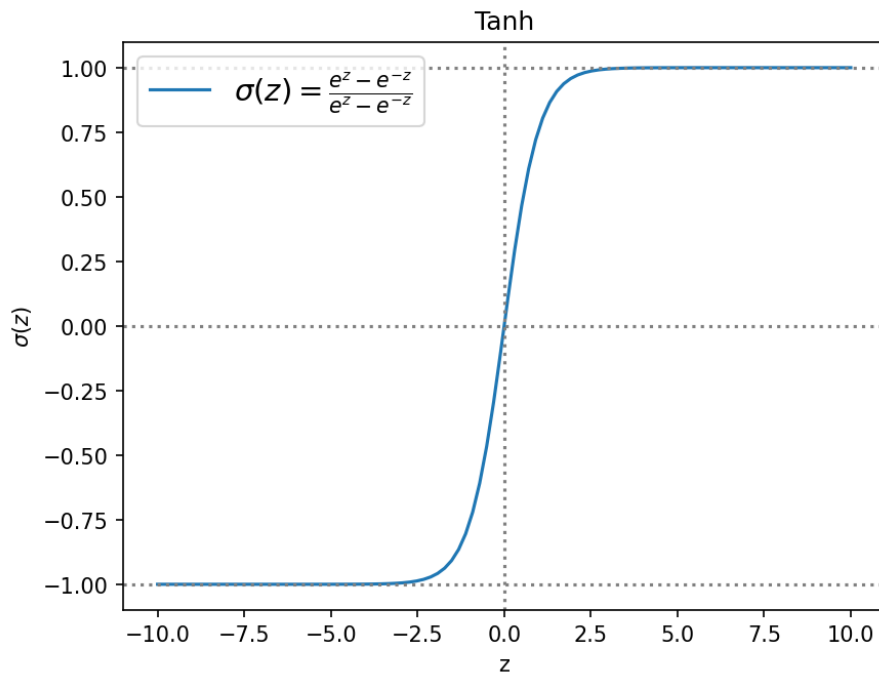


Figure 3.5 The hyperbolic tangent activation function.

Rectified Linear Unit (ReLU): The Rectified Linear Unit, or ReLU, has become the de facto standard for hidden layers in modern deep learning due to its simplicity and effectiveness,

$$\sigma(z) = \mathbf{max}(0, z). \quad (3.8)$$

It has the following properties: Nonlinearity, which makes it a piecewise linear function, leading to high nonlinearity.

Its advantages include: Computational efficiency: It is very fast to compute (it's a simple thresholding operation). Reduced slope of missing values: For positive inputs, the slope is fixed at 1, preventing missing values and accelerating training.

The disadvantage is the potential for a “dying ReLU” problem, where a neuron may stop working and permanently output zero if its weights are updated to always be negative.

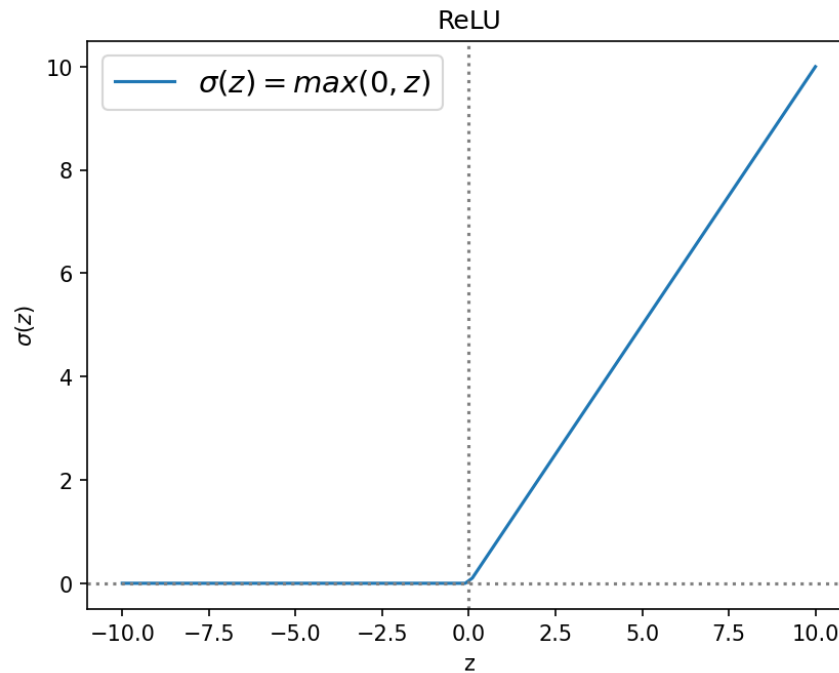


Figure 3.6 The ReLU activation function.

3.1 Training and Optimization

The core objective in training a neural networks is to determine the optimal set of weights (**W**) and biases (**b**) that enables the networks to accurately map input data to their corresponding outputs. This is achieved through an iterative optimization process that minimizes a predefined measure of error. This process fundamentally consists of three key components: the loss function, the optimization algorithm, and the backpropagation method for gradient computation.

3.1.1 The Loss Function

The primary objective of training a neural network is to iteratively adjust its internal parameters—the weights and biases—to progressively improve its predictive accuracy. To achieve this, we first need a precise, mathematical way to measure

how well the network is performing at any given moment. This is the role of the loss function (also known as a cost function or objective function), $\mathcal{L}$. It takes the network's predictions ($\hat{y}$) and the true target values (y) as input and outputs a single scalar value that quantifies the "error" or "loss." A smaller loss value signifies a better performance, while a larger value indicates a greater discrepancy between prediction and reality.

Conceptually, the loss function defines a high-dimensional "landscape" over the space of all possible network parameters. Each point on this landscape corresponds to a specific configuration of weights and biases, and its height represents the corresponding loss. The goal of the training process is, therefore, to navigate this complex terrain and find the coordinates of the lowest point—the global minimum—where the network's error is minimized. The choice of the loss function is a critical design decision, as it must accurately reflect the specific goals of the problem being solved. The choice of loss function depends on the nature of the task and there are two types:

Regression: A common choice is the Mean Squared Error (MSE), which calculates the average squared difference between the predicted and actual values,

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2, \quad (3.9)$$

where N is the number of samples in dataset.

Classification: The Categorical Cross-Entropy loss is typically used. It measures the dissimilarity between the predicted probability distribution and the true distribution of the classes,

$$\mathcal{L}_{CE} = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^C y_{ij} \log(\hat{y}_{ij}), \quad (3.10)$$

where C is the number of classes, and y_{ij} is 1 if sample i belongs to class j and 0 otherwise.

The goal of the training process is to find the network parameters that minimize this loss function.

3.2 Optimization via Gradient Descent

Once the prediction error has been quantified by the loss function $\mathcal{L}$, the next step is to systematically adjust the network's parameters, θ (representing all weights $\mathbf{W}$ and biases $\mathbf{b}$), to minimize this error. The foundational algorithm that powers this optimization process is Gradient Descent. The core principle is intuitive: to find the lowest point in the high-dimensional loss landscape, one should take iterative steps in the direction of the steepest descent.

The gradient of the loss function, $\nabla_{\theta}\mathcal{L}$, is a vector that points in the direction of the greatest increase in the loss. Therefore, to minimize the loss, we must update the parameters in the direction opposite to the gradient. The parameter update rule for each iteration is defined as

$$\theta_{new} = \theta_{old} - \eta \nabla_{\theta}\mathcal{L}, \quad (3.11)$$

where η is the learning rate, a critical hyperparameter that scales the size of each update step. The choice of learning rate significantly impacts the training process: a small learning rate results in slow but steady convergence, requiring many iterations to reach the minimum, while a large learning rate can speed up training but risks overshooting the minimum, potentially causing the loss to oscillate or even diverge.

3.2.1 Variants of Gradient Descent

In practice, computing the gradient over the entire dataset is inefficient. Therefore, Mini-batch Gradient Descent is the most common approach, striking a balance by computing the gradient over a small subset (a mini-batch) of the data at each step. This combines the stability of batch methods with the efficiency of Stochastic Gradient Descent (SGD).

3.2.2 Challenges in Deep Network Optimization

Training deep neural networks is challenging due to the highly non-convex nature of their loss landscapes, which are characterized by numerous local minima and saddle points. The training process is further complicated by two primary

gradient-related issues. The first is the vanishing gradient problem, where, particularly in deep architectures, the error signal shrinks exponentially as it is propagated backward, effectively halting learning in the network's earlier layers. Conversely, the training may suffer from exploding gradients, a phenomenon where the gradient signal grows uncontrollably, leading to large, unstable parameter updates and numerical overflow. These issues are typically mitigated using techniques such as careful weight initialization, normalization layers (e.g., Batch Normalization), and non-saturating activation functions like ReLU.

3.2.3 Advanced First-Order Optimization Algorithms

To navigate these complex loss landscapes more effectively, advanced optimization algorithms have been developed. These methods adapt the learning rate or incorporate past updates to accelerate convergence.

- **Momentum:** This method accelerates convergence by accumulating a velocity vector in directions of persistent gradient descent, helping to overcome small local minima and ravines.
- **Adam (Adaptive Moment Estimation):** This is one of the most widely used optimizers in deep learning. It combines the ideas of momentum with adaptive learning rates for each parameter, making it robust, efficient, and often a good default choice for a wide range of problems.

3.2.4 Second-Order Optimization for Scientific Machine Learning

The methods described above are first-order methods as they only utilize the gradient (the first derivative). For many scientific applications, including Physics-Informed Neural Networks (PINNs), higher precision is often required. This motivates the use of second-order optimization methods, which incorporate curvature information via the Hessian matrix (the second derivative).

While the pure form, Newton's method, is computationally intractable for large networks due to the need to compute and invert the Hessian matrix, quasi-Newton methods provide a practical alternative. The most prominent among these is

the L-BFGS (Limited-memory Broyden–Fletcher–Goldfarb–Shanno) algorithm. L-BFGS approximates the inverse Hessian matrix using information from previous gradient evaluations, allowing it to take more effective and direct steps toward a minimum. Because of its ability to achieve high-precision solutions, L-BFGS is frequently the optimizer of choice for training PINNs, especially in the final stages of fine-tuning the model to satisfy the underlying physical laws expressed as differential equations.

3.3 The Backpropagation Algorithm

The optimization algorithms described previously, such as Gradient Descent, all rely on one crucial piece of information: the gradient of the loss function with respect to every network parameter ($\nabla_{\theta}\mathcal{L}$). For a deep neural network with millions or even billions of parameters, calculating these gradients directly would be an astronomically complex and computationally prohibitive task. The Backpropagation algorithm is the elegant and highly efficient solution to this problem. It is not an optimization algorithm itself, but rather the cornerstone technique used to compute the necessary gradients that feed into the optimizer.

At its core, backpropagation is a clever and practical application of the chain rule from differential calculus. It allows the network to determine how much each individual weight and bias contributed to the final output error, and to do so in a computationally tractable manner. The algorithm operates in two distinct phases: a forward pass and a backward pass.

3.3.1 The Forward Pass: From Input to Prediction

The first phase is the forward pass, which is simply the process of making a prediction. An input vector, $\mathbf{x}$, is fed into the first layer of the network. The output of this layer, after applying the weighted sum and activation function, becomes the input for the next layer. This process is repeated sequentially, layer by layer, until the final layer produces the network’s output, $\hat{y}$.

During this pass, the intermediate values (the weighted sums and the

activations) at each neuron are stored in memory. These values are crucial for the subsequent backward pass. Once the forward pass is complete, the loss function, $\mathcal{L}(\hat{y}, y)$, is computed to quantify the error of the prediction.

Input (x) → Layer 1 → Layer 2 → ⋯ → Layer L → Output ($\hat{y}$) → Loss ($\mathcal{L}$).

3.3.2 The Backward Pass

The second phase is the backward pass, where the magic of backpropagation happens. This phase propagates the error signal backward through the network, from the final layer to the first.

1. **Initialization:** The process begins at the very end of the network. The algorithm first computes the derivative of the loss function with respect to the output of the final layer ($\frac{\partial \mathcal{L}}{\partial \hat{y}}$). This initial gradient tells us how the loss changes with respect to the final prediction.
2. **Recursive Gradient Calculation:** Using the chain rule, the algorithm then works its way backward, one layer at a time. For each layer, it calculates the gradient of the loss function with respect to that layer's parameters (its weights and biases). To do this, it uses the gradient from the layer immediately ahead of it and the intermediate values (activations) that were stored during the forward pass.

For a weight $W_{ij}^{(l)}$ connecting neuron i in layer $l-1$ to neuron j in layer l , the gradient is computed by applying the chain rule:

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(l)}} = \frac{\partial \mathcal{L}}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial W_{ij}^{(l)}} \quad (3.12)$$

Where $z_j^{(l)}$ is the weighted sum input to neuron j . The term $\frac{\partial \mathcal{L}}{\partial z_j^{(l)}}$ is the error signal propagated from the next layer, and $\frac{\partial z_j^{(l)}}{\partial W_{ij}^{(l)}}$ is simply the activation from the previous layer, $a_i^{(l-1)}$. This process is repeated until the gradients for all parameters in the entire network have been computed.

The invention and application of the backpropagation algorithm were a watershed moment in the history of artificial intelligence. It transformed the training

of deep, multi-layered neural networks from a theoretical impossibility into a practical reality, directly enabling the deep learning revolution we see today.

3.3.3 Automatic Differentiation

While backpropagation is often presented as a standalone algorithm, it can be formally understood as an application of reverse-mode automatic differentiation (AutoDiff). Automatic differentiation differs from symbolic and numerical differentiation in that it computes derivatives exactly up to machine precision, while maintaining computational complexity comparable to the evaluation of the original function itself.

AutoDiff works by decomposing a complex function into a sequence of elementary operations, each with a known derivative. By systematically applying the chain rule, derivatives of the entire function can be computed efficiently. Two primary modes of AutoDiff exist: forward mode, which propagates derivatives from inputs to outputs, and reverse mode, which propagates derivatives from outputs back to inputs. Neural network training, which involves computing the gradient of a scalar loss with respect to millions of parameters, is ideally suited to reverse-mode AutoDiff.

Thus, backpropagation can be regarded not as a separate algorithm, but as the specific implementation of reverse-mode AutoDiff for neural networks. This interpretation emphasizes that the efficiency and feasibility of training modern deep networks fundamentally rely on automatic differentiation.

3.4 Types of Neural Networks

Neural networks exist in a variety of architectures, each tailored to different problem domains. The most fundamental form is the Multilayer Perceptron (MLP), also referred to as a fully connected feedforward network. In an MLP, every neuron in one layer is connected to every neuron in the next layer, making it suitable for approximating generic nonlinear functions. Despite its generality, the MLP does not efficiently exploit specific data structures, such as spatial or temporal correlations.

To address these limitations, specialized architectures have been developed: Convolutional Neural Networks (CNNs): CNNs incorporate convolutional layers that apply filters across local receptive fields. This architecture is particularly effective for spatially structured data such as images, where local features can be hierarchically combined into high-level representations.

Recurrent Neural Networks (RNNs): RNNs are designed for sequential data by incorporating recurrent connections that maintain hidden states across time steps. Variants such as Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRUs) mitigate issues of vanishing and exploding gradients, enabling modeling of long-range dependencies in sequences.

Transformers: More recently, transformer architectures have emerged, leveraging self-attention mechanisms instead of recurrence or convolution. Transformers excel in capturing global dependencies and have achieved state-of-the-art performance in natural language processing and other sequential modeling tasks.

Although the present work focuses primarily on fully connected architectures, an understanding of these diverse network types highlights the adaptability of neural networks to different domains, including applications in scientific machine learning.

3.5 Regularization in Neural Networks

A critical aspect of training neural networks is ensuring that the learned model generalizes well to unseen data. Overparameterized models are prone to overfitting, where they memorize training examples rather than learning generalizable patterns. Regularization techniques are introduced to mitigate overfitting by constraining the effective capacity of the model. Key strategies include:

- **Weight Decay (L2 Regularization):** This technique adds a penalty term to the loss function that is proportional to the squared magnitude of the weights. By discouraging excessively large parameter values, it promotes simpler models.
- **Dropout:** During training, a random subset of neurons is temporarily deactivated or "dropped out" at each iteration. This forces the network to learn

more robust and redundant representations, preventing over-reliance on any single neuron.

- **Batch Normalization:** This method normalizes the activations of intermediate layers, which helps to reduce internal covariate shift. This stabilizes the training process and has been shown to provide an implicit regularization effect.
- **Early Stopping:** Training is monitored on a separate validation set, and the process is terminated once the validation error ceases to improve, even if the training error is still decreasing. This directly prevents the model from overfitting to the training data.

These strategies are widely adopted in practice and serve as essential tools in maintaining the critical balance between model complexity and generalization performance.

3.6 Challenges in Neural Network Training

Training neural networks involves navigating complex, high-dimensional optimization landscapes, which introduces several challenges:

- **Vanishing and Exploding Gradients:** In deep networks, gradients propagated backward through many layers can shrink or grow exponentially, hindering effective parameter updates. Techniques such as careful initialization, gradient clipping, and architectural innovations (e.g., residual connections) have been developed to address this issue.
- **Overfitting:** Due to their large capacity, neural networks may overfit small datasets. Regularization methods, data augmentation, and larger training datasets are common countermeasures.
- **Non-convex Loss Landscapes:** The optimization problem is highly non-convex, containing numerous local minima and saddle points. While stochastic gradient methods often find acceptable solutions, convergence to globally optimal minima is not guaranteed.
- **Computational Complexity:** Training large-scale networks requires significant computational resources. Advances in GPU/TPU hardware and

parallelization techniques have played a central role in the success of deep learning.

These challenges highlight the need for robust optimization techniques and careful model design to achieve effective learning.

3.7 Applications and Transition to Physics-Informed Neural Networks

Neural networks have demonstrated remarkable success across diverse application domains. In computer vision, CNNs underpin image recognition, object detection, and medical imaging. In natural language processing, transformer-based architectures power state-of-the-art machine translation and language modeling systems. In reinforcement learning, neural networks have enabled agents to master complex tasks such as game playing and robotic control.

Beyond traditional machine learning applications, neural networks are increasingly applied to scientific domains. They serve as surrogate models for expensive simulations, accelerators for numerical solvers, and tools for discovering patterns in high-dimensional experimental data. One of the most promising directions is the use of neural networks to solve differential equations, either as universal function approximators or in hybrid frameworks that combine data-driven learning with known physical laws.

This transition leads naturally to the concept of Physics-Informed Neural Networks (PINNs). PINNs extend conventional neural networks by embedding governing equations directly into the training process, enabling solutions that respect both data and physical constraints. Thus, PINNs represent a bridge between traditional neural network methods and scientific modeling, and form the foundation of the subsequent chapter of this thesis.

CHAPTER IV

PHYSICS-INFORMED NEURAL NETWORKS (PINNs)

In this chapter, we present the framework of Physics-Informed Neural Networks (PINNs), a powerful class of machine learning models that incorporate physical laws directly into the training process of neural networks. Unlike conventional neural networks, which are trained solely on observational or simulated data, PINNs integrate governing equations such as ordinary and partial differential equations into the optimization objective. This enables the network not only to fit available data but also to respect the fundamental dynamics of the underlying physical system.

The key idea of PINNs is to embed the residuals of the governing equations into the loss function. By leveraging automatic differentiation, neural networks can provide exact derivatives of their outputs with respect to inputs, allowing differential operators to be imposed efficiently during training. This dual nature of PINNs, combining data-driven learning with physics-based constraints, makes them particularly attractive for problems in scientific machine learning where data may be sparse, expensive to obtain, or incomplete.

In the context of cosmology, and in particular in the study of axion dynamics, PINNs offer a novel approach to solving the coupled nonlinear equations that describe the evolution of scalar fields and the cosmic expansion. Traditional numerical solvers often face challenges such as stiffness, high computational cost, and limited flexibility when extending to multi-field scenarios. PINNs, on the other hand, provide a flexible framework capable of learning solutions across wide parameter spaces while inherently incorporating physical consistency.

4.1 Origin of Physics-Informed Neural Networks

The idea of using neural networks as function approximators for scientific problems dates back several decades. Early theoretical work established that

feedforward neural networks are universal approximators capable of representing any continuous function to arbitrary accuracy on compact domains [27, 28]. Building on this foundation, Lagaris et al. and Karniadakis et al. demonstrated one of the first applications of neural networks to the solution of differential equations, where the network was trained to satisfy boundary conditions and differential operators directly [29, 17]. This work laid the groundwork for later developments that would systematically integrate physical laws into the learning process.

The concept of Physics-Informed Neural Networks (PINNs) was formalized two decades later by Raissi, Perdikaris, and Karniadakis [30, 31]. PINNs introduced a unified framework for solving forward and inverse problems governed by ordinary and partial differential equations. The central innovation was the inclusion of the residual of the governing equations directly in the loss function of neural networks, enforced through automatic differentiation. This approach allowed PINNs to leverage both data and physics simultaneously, making them particularly effective in regimes where observational data are scarce or noisy.

Since their introduction, PINNs have rapidly gained traction across a wide spectrum of scientific and engineering disciplines. Applications range from fluid dynamics [31], solid mechanics [32], quantum mechanics [33], general relativity [19, 20], astrophysics [34, 35, 36, 37, 38, 39], and any examples used in physics, to emerging areas in cosmology and high-energy physics. Their flexibility, data efficiency, and ability to integrate prior knowledge make PINNs a promising tool for scientific machine learning, and particularly suitable for studying nonlinear and multi-scale phenomena such as those arising in axion cosmology.

4.2 Structure of Physics-Informed Neural Networks

A Physics-Informed Neural Network (PINNs) extends the standard neural network framework by embedding physical knowledge directly into its training objective. The core structure of a PINNs consists of the following components.

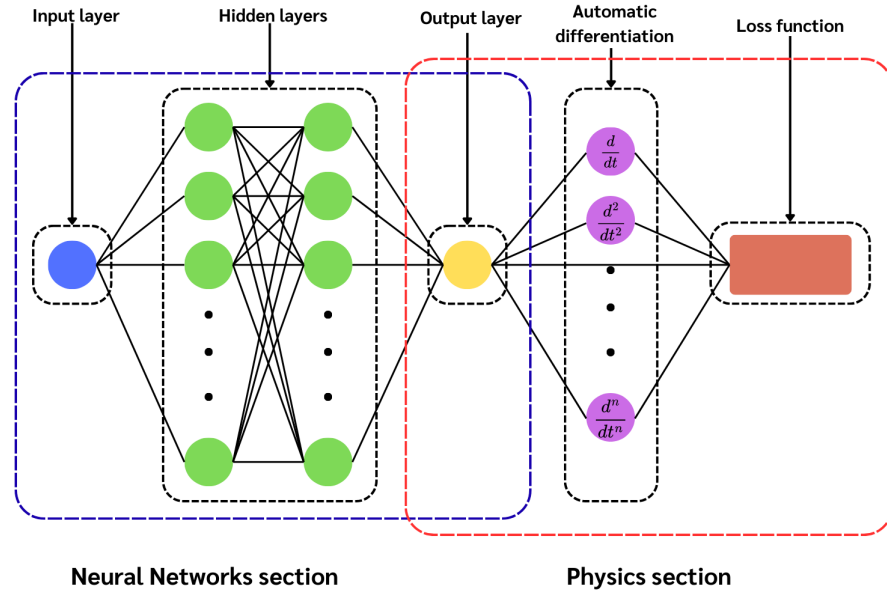


Figure 4.1 Architecture of the PINNs model to used in this work.

4.2.1 Neural Network architecture

The network employed in PINNs is typically a fully connected feedforward neural network (also called a multilayer perceptron). The input layer receives the independent variables of the problem, such as spatial coordinates $\mathbf{x}$ and temporal coordinate t . The output layer produces the dependent variables of interest, such as a scalar field $u(\mathbf{x}, t)$ or multiple coupled physical quantities. Formally, the network defines a mapping,

$$f_{\theta} : (\mathbf{x}, t) \mapsto u_{\theta}(\mathbf{x}, t), \quad (4.1)$$

where θ represents the learnable weights and biases.

4.2.2 Residual points and collocation sampling

To enforce the governing equations, PINNs introduce a set of collocation points $\{(\mathbf{x}_r^{(i)}, t_r^{(i)})\}_{i=1}^{N_r}$ in the domain of interest. At these points, the residual of the differential operator is evaluated. The choice of collocation points is critical: they can be distributed uniformly, drawn from random sampling, or adaptively refined in regions of high error.

4.2.3 Loss function components

The total loss function of a PINNs typically includes multiple contributions,

$$\mathcal{L}(\theta) = \lambda_{data} \mathcal{L}_{data}(\theta) + \lambda_{phys} \mathcal{L}_{phys}(\theta) + \lambda_{bc/ic} \mathcal{L}_{bc/ic}(\theta), \quad (4.2)$$

where

- $\mathcal{L}_{data}$ enforces agreement with available data or observations,
- $\mathcal{L}_{phys}$ enforces the residual of the governing equations (e.g., ODEs or PDEs),
- $\mathcal{L}_{bc/ic}$ enforces boundary or initial conditions,
- $\lambda_{data}, \lambda_{phys}, \lambda_{bc/ic}$ are weighting coefficients balancing the contributions.

For example, if the governing equation is expressed as $\mathcal{N}[u] = 0$, the physics-informed term becomes

$$\mathcal{L}_{phys}(\theta) = \frac{1}{N_r} \sum_{i=1}^{N_r} |\mathcal{N}[u_\theta](\mathbf{x}_r^{(i)}, t_r^{(i)})|^2. \quad (4.3)$$

4.2.4 Automatic differentiation for physics constraints

A key element in the structure of PINNs is the use of automatic differentiation (AutoDiff) to compute the necessary derivatives of the neural network output with respect to inputs. This enables exact computation (up to machine precision) of differential operators without resorting to numerical discretization, which is especially advantageous for enforcing higher-order derivatives in partial differential equations.

4.2.5 Training Procedure

Training a PINNs involves minimizing the total loss $\mathcal{L}(\theta)$ using gradient-based optimization algorithms. In this work, a two-stage strategy is employed: starting with an Adam optimizer to explore the parameter space, followed by a quasi-Newton method such as L-BFGS to refine the convergence to a high-precision solution. This hybrid approach is particularly effective in overcoming the stiffness and poor conditioning problems commonly encountered in PINNs training. The following figure 4.2 is a flowchart of all process of PINNs used in this work.

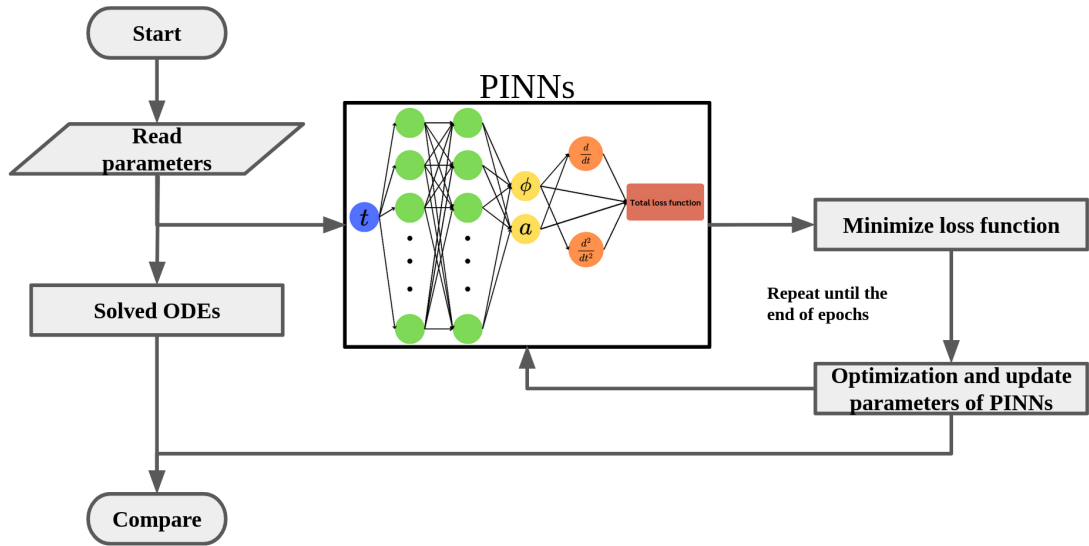


Figure 4.2 Flowchart showing the workflow and calculations for this task. This flowchart also includes the PINNs training process.

4.3 Application of PINNs to Axion Cosmology

Having introduced the general structure of Physics-Informed Neural Networks, we now turn to their application in the context of cosmology, with a specific focus on the dynamics of axion fields. Axions, originally proposed in the context of the strong CP problem in quantum chromodynamics, also arise naturally in string theory compactifications as light scalar fields. In cosmology, axion-like particles are compelling candidates for dark matter and play a crucial role in models of the early universe. Their evolution is governed by coupled nonlinear ordinary differential

equations (ODEs), involving both the scalar field dynamics and the cosmological expansion rate.

4.3.1 Governing Equations of Axion Dynamics

The evolution of a homogeneous axion field $\phi(t)$ in an expanding Friedmann–Lemaître–Robertson–Walker (FLRW) universe is described by the Klein–Gordon equation,

$$\ddot{\phi} + 3H\dot{\phi} + m_a^2\phi = 0, \quad (4.4)$$

where $H(t) = \dot{a}(t)/a(t)$ is the Hubble parameter and the scale factor $a(t)$ evolves according to the Friedmann equation,

$$H^2 = \frac{1}{3}(\rho_\phi + \rho_m + \rho_r + \rho_\Lambda), \quad (4.5)$$

where $\rho_\phi = \frac{1}{2}\dot{\phi}^2 + \frac{1}{2}m_a^2\phi^2$ is the axion energy density. In multi-field scenarios, the Klein–Gordon equation generalizes to an interacting system of scalar fields coupled through their potential and the expansion rate.

In this study, we consider a universe composed of matter, radiation, and a cosmological constant. Within the framework of early-universe axion cosmology, particularly scenarios governed by the misalignment mechanism (see Refs. [5, 40, 41]), the axion field is initialized with a displacement ($\phi(t_0) = \phi_i$ from the minimum of its potential, representing a random field value following Peccei–Quinn symmetry breaking. The initial velocity is set to $\dot{\phi}(t_0) = 0$, since at very early times the Hubble expansion rate satisfies $H \gg m_a$, producing strong friction that effectively “freezes” the field. In this overdamped regime, the axion remains nearly constant until the Hubble parameter decreases to $H \sim m_a$, at which point the field begins to oscillate.

For the scale factor, we initialize with a small value $a(t_0) = a_0 \ll 1$, consistent with early-time cosmology, and determine $\dot{a}(t_0)$ from the Friedmann equation to ensure compatibility with the total initial energy density (including contributions from radiation, matter, and the scalar field). These initial conditions are standard in numerical studies of axion evolution and are sufficient to capture the key dynamics of the frozen and oscillatory regimes.

All quantities are expressed in reduced Planck units, where both the reduced Planck mass M_{Pl} and the Hubble mass scale M_{H} are set to unity. The specific initial conditions used for the integration are

$$a(0) = 10^{-8}, \quad \phi(0) = 1.0, \quad \dot{\phi}(0) = 0, \quad (4.6)$$

with background energy densities given by

$$\rho_m = 0.81, \quad \rho_r = 0.00027138, \quad \rho_\Lambda = 2.19, \quad (4.7)$$

where ρ_m , ρ_r , and ρ_Λ denote the densities of matter, radiation, and the cosmological constant, respectively.

The system is numerically integrated over the interval $t \in [10^{-8}, 1]$ on a logarithmically spaced grid. This setup represents the conventional numerical approach, enabling resolution of early-time dynamics and accurate tracking of the axion field's evolution and its impact on the expansion history of the Universe. The resulting solution serves as a benchmark reference for assessing the performance of the PINNs-based method.

4.3.2 PINNs Formulation for Axion Cosmology

To apply PINNs to this problem, we define the neural network outputs as the cosmological quantities of interest. For example, the network can be designed to approximate the scalar field $\phi_p(t)$ and scale factor $a_p(t)$, given cosmic time t as input. The governing equations are then enforced by including their residuals in the loss function,

$$\mathcal{L}_{\text{phys}} = \frac{1}{N} \sum_{i=1}^N \left[\ddot{\phi}_p(t_i) + \sqrt{3} (\rho_{\phi,p}(t_i) + \rho_{\text{tot},p})^{1/2} \dot{\phi}_p(t_i) + m_a^2 \phi_p(t_i) \right]^2 \quad (4.8)$$

$$+ \frac{1}{N} \sum_{i=1}^N \left[\dot{a}_p(t_i) - \frac{1}{\sqrt{3}} (a_p^2(t_i) \rho_{\phi,p} + a_p^2(t_i) \rho_{\text{tot},p})^{1/2} \right]^2, \quad (4.9)$$

where $\rho_{\phi,p}(t_i) = \frac{1}{2} \dot{\phi}_p^2(t_i) + \frac{1}{2} m_a^2 \phi_p^2(t_i)$ and $\rho_{\text{tot},p} = \rho_m/a_p^3(t_i) + \rho_r a_p^4(t_i) + \rho_\Lambda$

4.3.3 Boundary and Initial Conditions

The axion field evolution depends sensitively on its initial conditions. In the PINNs framework, these are imposed as additional loss terms,

$$\mathcal{L}_{ic} = \lambda_a [a_p(t_0) - a(t_0)]^2 + \lambda_\phi [\phi_p(t_0) - \phi(t_0)]^2, \quad (4.10)$$

4.4 Integrating Fourier Features and Residual Block into PINNs

Fourier features (Sinusoidal positional encoding) is one of the solutions for neural networks, especially MLPs. MLPs are often limited in their ability to learn functions with high-frequency components, such as waveforms, oscillations, or oscillatory PDEs. Activation functions like ReLU or Tanh are biased towards learning low-frequency functions (called spectral bias). Fourier features convert scalar or vector inputs into multi-frequency spaces to help neural networks learn oscillating functions and overcome gradient collapse problems when the input is in a narrow range (e.g., $[0, 1]$). This allows neural networks to learn functions with high latency or complexity better than conventional linear encoding,

$$\phi(t) = ([\sin(2\pi Bt)], [\cos(2\pi Bt)]). \quad (4.11)$$

Residual Block (ResNet block) is a residual learning method, rather than a direct mapping method, which reduces the vanishing gradient problem and speeds up training in deep networks,

$$\text{Output} = F(x) + x, \quad (4.12)$$

where $F(x) = \mathbf{W}_2 \cdot \sigma(\mathbf{W}_1 \cdot x)$, x is a input, $\mathbf{W}_1, \mathbf{W}_2$ is the weight matrices of the linear layer and σ is activation function.

The combination of Fourier features and residual block will provide neural networks that can accurately learn complex nonlinear oscillation temporal functions (e.g., scale factors, scalar field dynamics). This is achieved through Fourier features that capture oscillation characteristics, learn high-frequency functions well, and residual block that can reduce vanishing gradients and learn deeper without degradation.

By using Fourier features and residual blocks in neural networks, we can capture the behavior of axion scalar fields that have high frequency and very fast oscillations when we increase the mass of the axion scalar field (m_a), which will make our PINNs learn faster, thus reducing the learning time.

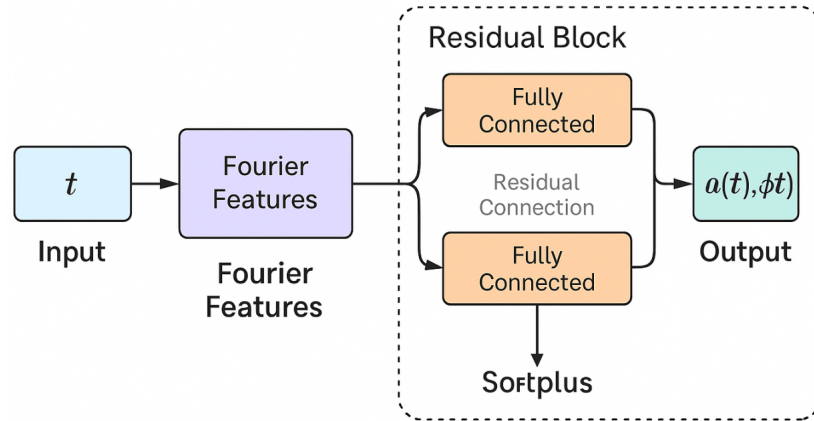


Figure 4.3 The architecture of the PINNs has been improved by adding Fourier features and residual block into the neural networks structure.

4.5 ODEs Solver

In the study of axion cosmology in the olden days Will use the solution by using the numerical method. At this work, we will use the SciPy package, which is called `scipy.solve_ivp` with the Runge–Kutta (RK45) method [42, 43, 44] as the baseline solver for comparison with PINNs.

4.6 Hyperparameter tuning

In this section, we want to present a tuning method for PINNs so that our PINNs can learn quickly. The hyperparameter tuning in this work involves scanning for the best fit of the hyperparameters using something called Mean Absolute Error (MAE), which is how we compare the results of two methods, PINNs and ODE,

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |\phi_p(t_i) - \phi_{\text{ode}}(t_i)|, \quad (4.13)$$

where $\phi_p(t_i)$ and $\phi_{\text{ode}}(t_i)$ denote the predictions from the PINNs and the ODEs solver at time t_i , respectively, and N is the number of evaluation points.

CHAPTER V

RESULTS AND DISCUSSION

In this chapter, we are going to present our results on Physics-Informed Neural Networks (PINNs) compare with ODE solvers, the combination of PINNs with Fourier features and residual blocks, and hyperparameter values of this work. We will divide the results into 2 types: with and without Fourier features and residual blocks.

5.1 PINNs without Fourier features and residual blocks

This is the result of using PINNs by comparison between ODEs and PINNs for $\phi(t)$ and $a(t)$ with axion scalar field mass $m_a = 10$, which gives the result as shown in the following figure.

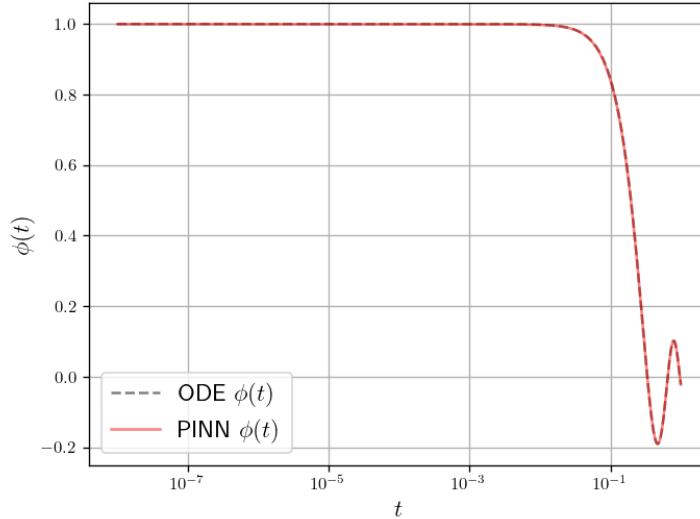


Figure 5.1 Comparison between ODEs and PINNs for $\phi(t)$ at cosmic time in co-moving coordinates within the FLRW background, which axion scalar field mass $m_a = 10$.

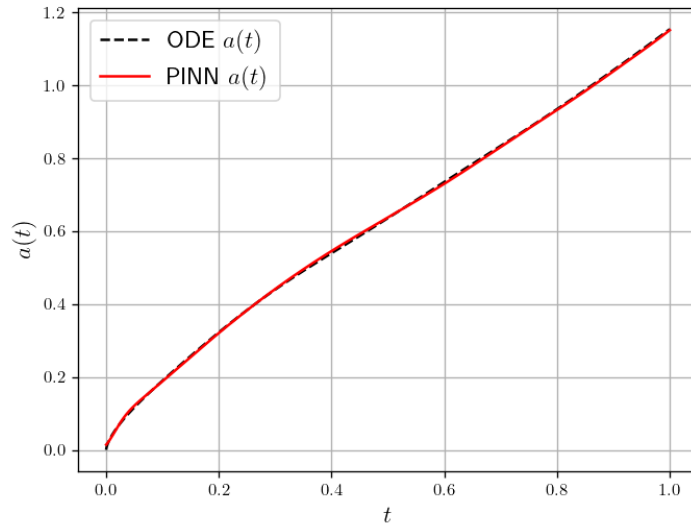


Figure 5.2 Comparison between ODEs and PINNs for $a(t)$ in linear scale with scalar field mass $m_a = 10$.

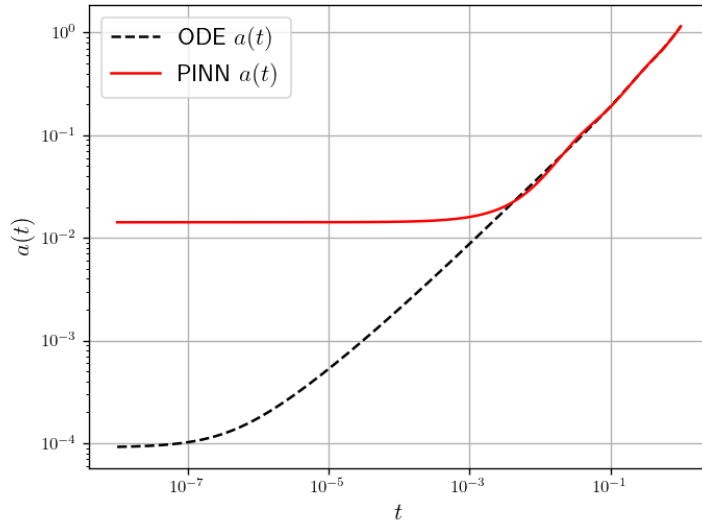


Figure 5.3 Comparison between ODEs and PINNs for $a(t)$ in linear scale with scalar field mass $m_a = 10$. This graph presents the comparison of the scale factor $a(t)$ in logarithmic scale. Although the agreement remains very good at late times, the logarithmic plot highlights noticeable deviations during the early universe regime. This arises because, at the onset of cosmic evolution, the scale factor assumes extremely small values often several orders of magnitude below unity.

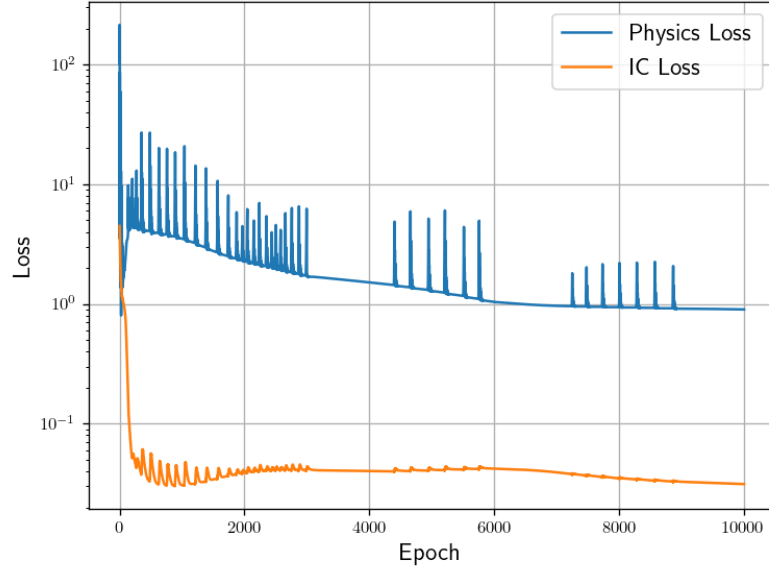


Figure 5.4 The evolution of leaning curve during training with Adam optimizer for baseline models trained on scalar field dynamics with scalar field mass $m_a = 10$.

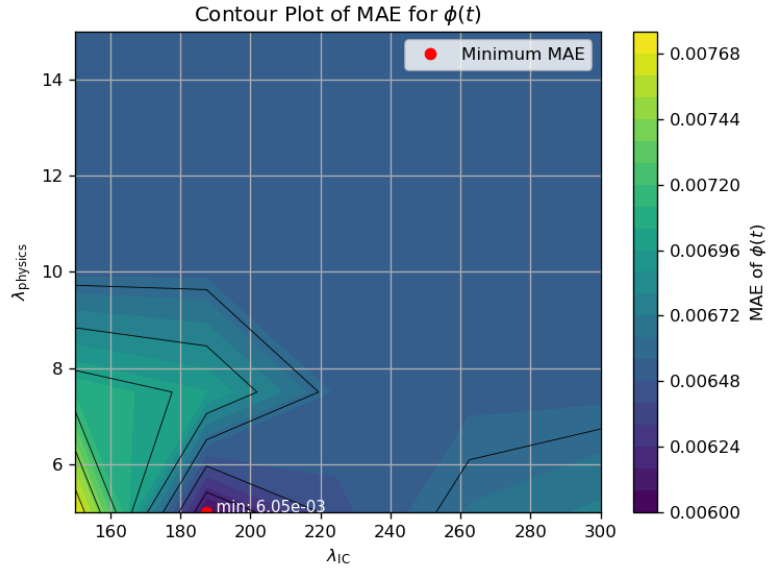


Figure 5.5 Contour plots of the mean absolute error (MAE) for $\phi(t)$ with scalar field mass $m_a = 10$. This is the hyperparameter value obtained from scanning the MAE values for λ_{IC} and λ_{phys} , which yielded the minimum values of the hyperparameters are approximately 6.05×10^{-3} occurs near $\lambda_{IC} = 190$, $\lambda_{phys} = 5$.

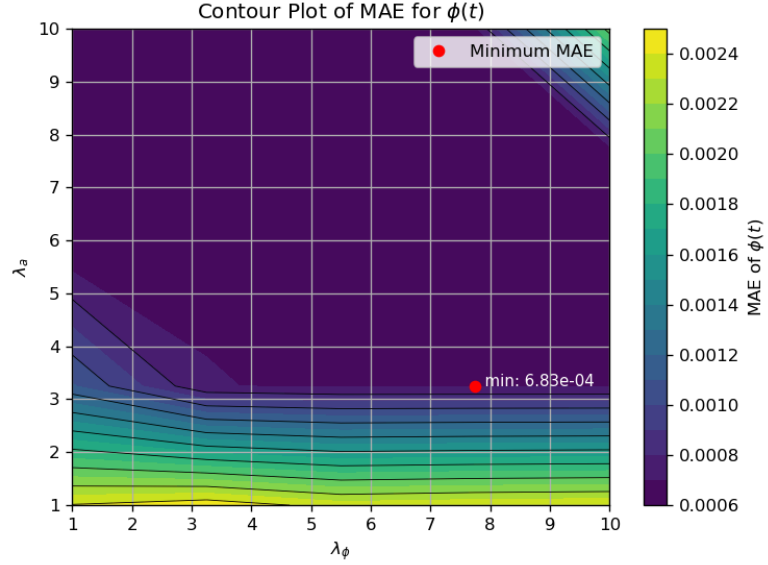


Figure 5.6 Contour plots of the mean absolute error (MAE) for $\phi(t)$ with scalar field mass $m_a = 10$. The MAE of λ_a and λ_ϕ at the minimum point approximately 6.83×10^{-4} at $\lambda_a = 3$, $\lambda_\phi = 8$.

5.2 PINNs with Fourier features and residual blocks

This is the result of using PINNs with Fourier features and residual blocks, tested by adding small amounts of mass. We focus on the $\phi(t)$ value rather than the $a(t)$ value, which shows that the following a values may be slightly off. The results are shown in the following figure.

We compare these results to our previous findings for the $m_a = 10$ case to assess whether incorporating an Fourier features and residual blocks into the PINNs framework resolves the issues with the scale factor at early times. As shown in the figure below.

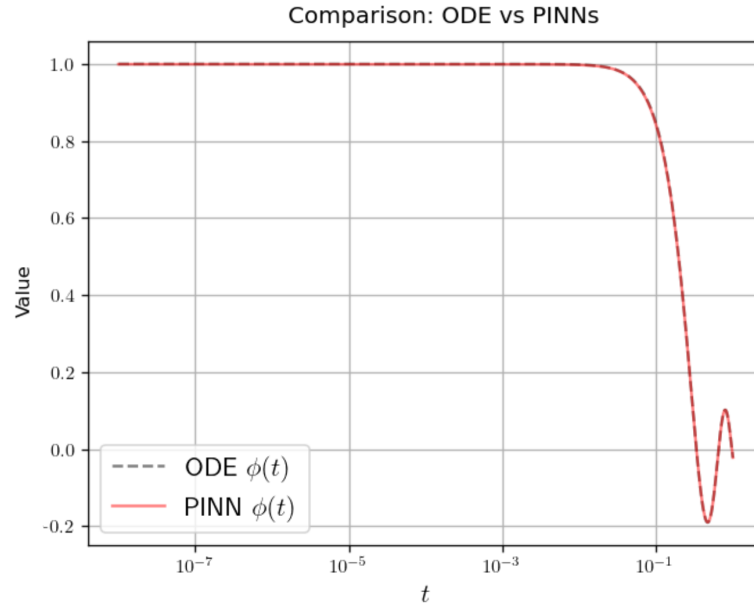


Figure 5.7 Comparison between ODEs and PINNs for $\phi(t)$ with scalar field mass $m_a = 10$ after adding Fourier features and residual blocks.

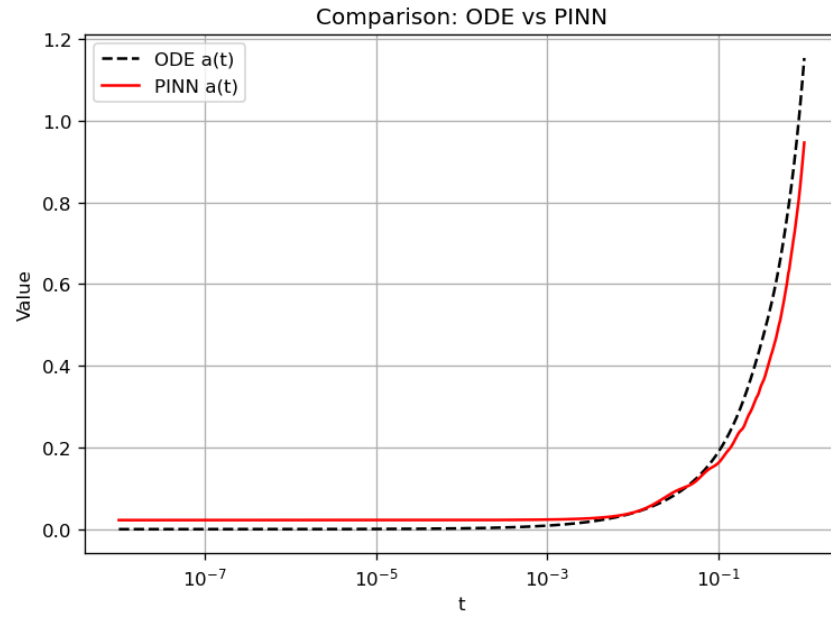


Figure 5.8 Comparison between ODEs and PINNs for $a(t)$ with scalar field mass $m_a = 10$. It was found that the results after adding Fourier features and residual blocks were much more different from the ODEs.

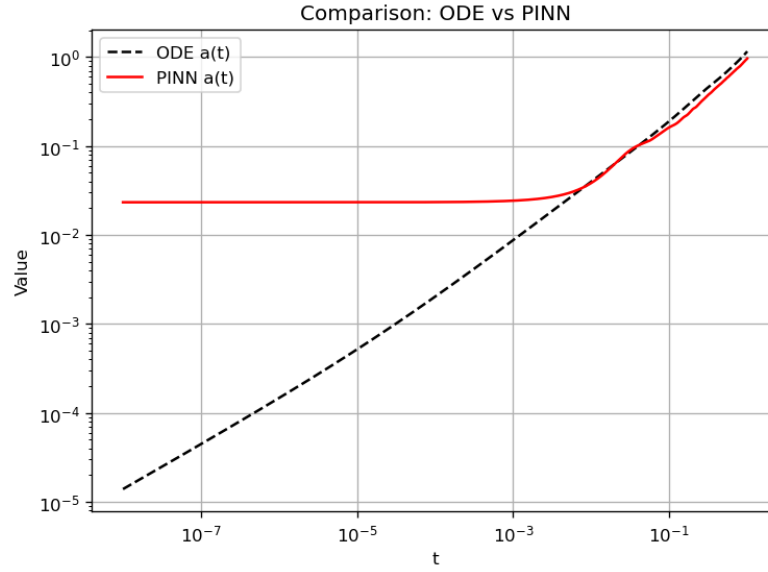


Figure 5.9 Comparison between ODEs and PINNs for $a(t)$ in linear scale with scalar field mass $m_a = 10$. This graph presents the comparison of the scale factor $a(t)$ in logarithmic scale. The consistency is found to remain good at late times, but not as good as the case without Fourier features and residual blocks. The logarithmic graph shows that the deviations that appeared in the early universe are still very different.

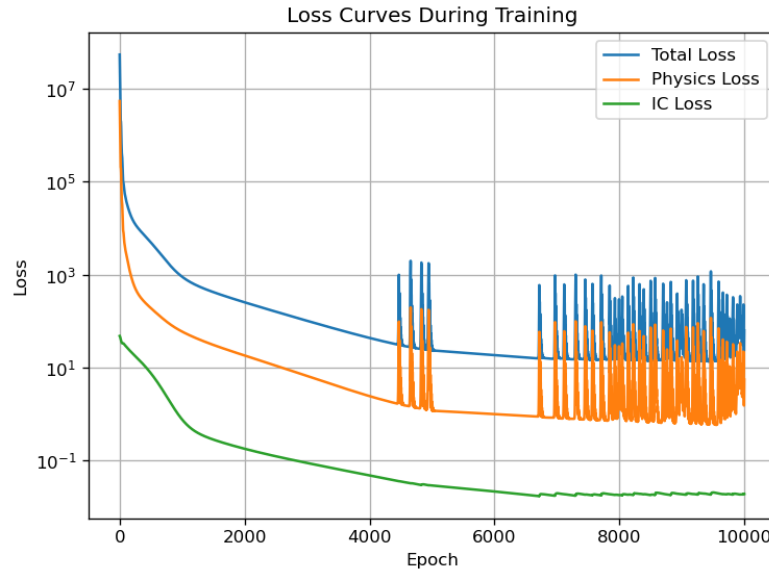


Figure 5.10 The evolution of leaning curve during training with Adam optimizer for baseline models trained on scalar field dynamics with scalar field mass $m_a = 10$. We can see that the value of the learning curve has increased more than before adding Fourier features and residual blocks.

This result still did not yield an optimal solution for the scale factor $a(t)$ at earlier time. Therefore, we will now shift our focus to the scalar field $\phi(t)$.

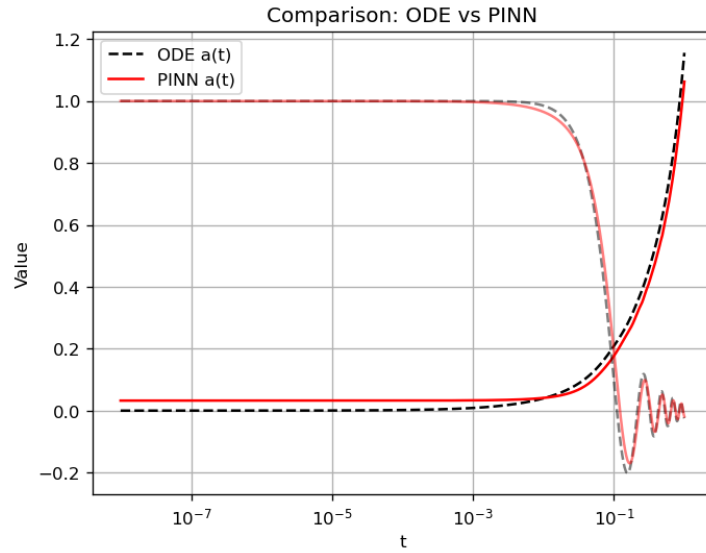


Figure 5.11 Comparison between ODEs and PINNs for $\phi(t)$ (oscillation line) and $a(t)$ (exponential line) with scalar field mass $m_a = 30$.

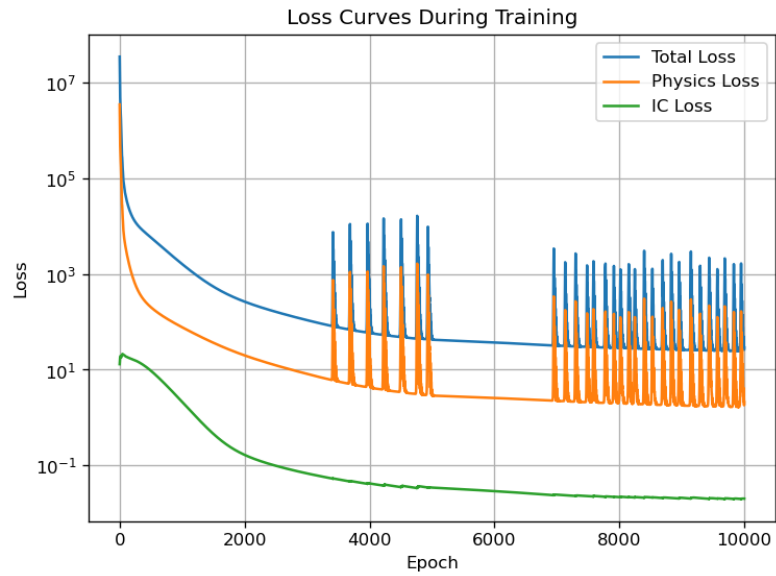


Figure 5.12 The evolution of leaning curve during training with Adam optimizer for baseline models trained on scalar field dynamics with scalar field mass $m_a = 30$.

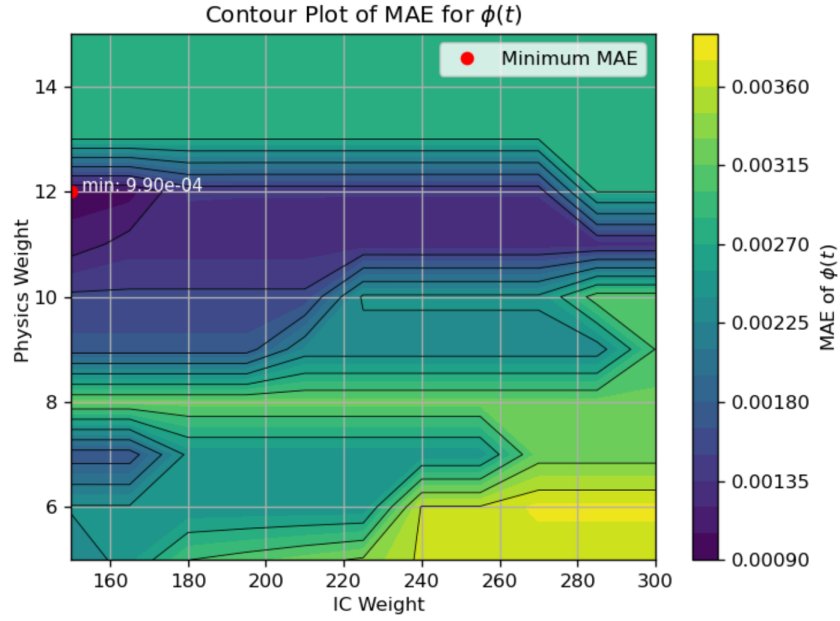


Figure 5.13 Contour plots of the mean absolute error (MAE) for $\phi(t)$ with scalar field mass $m_a = 10$. This is the hyperparameter value obtained from scanning the MAE values for λ_{IC} and λ_{phys} , which yielded the minimum values of the hyperparameters are approximately 9.90×10^{-4} occurs near $\lambda_{IC} = 150$, $\lambda_{phys} = 12$.

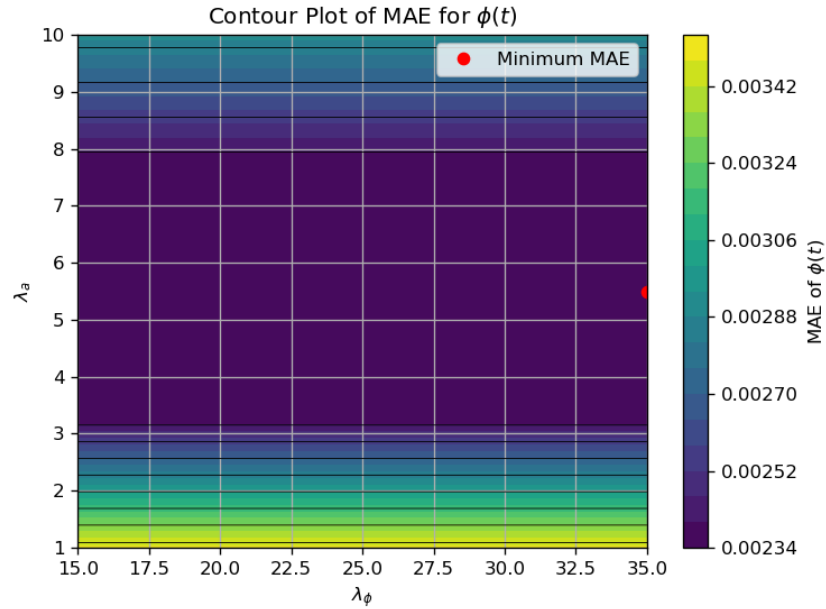


Figure 5.14 Contour plots of the mean absolute error (MAE) for $\phi(t)$ with scalar field mass $m_a = 30$. The MAE of λ_a and λ_ϕ at the minimum point approximately 2.34×10^{-3} at $\lambda_a = 5.5$, $\lambda_\phi = 35.0$.

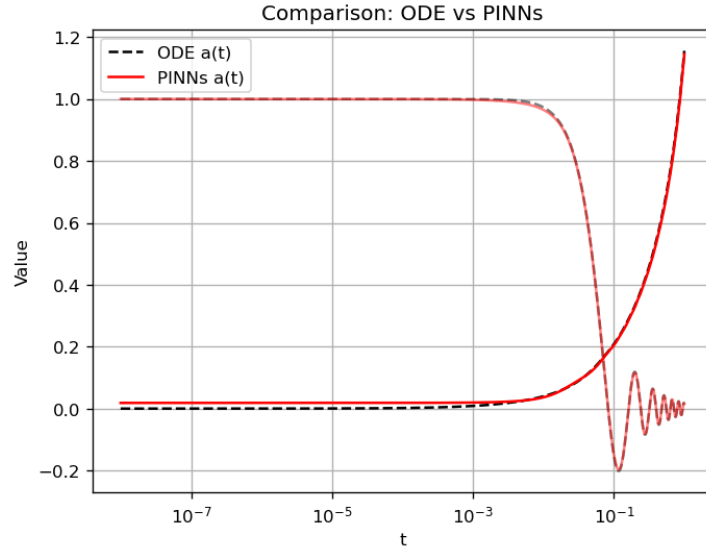


Figure 5.15 Comparison between ODEs and PINNs for $\phi(t)$ (oscillation line) and $a(t)$ (exponential line) with scalar field mass $m_a = 40$, using the same hyperparameter as $m_a = 30$.

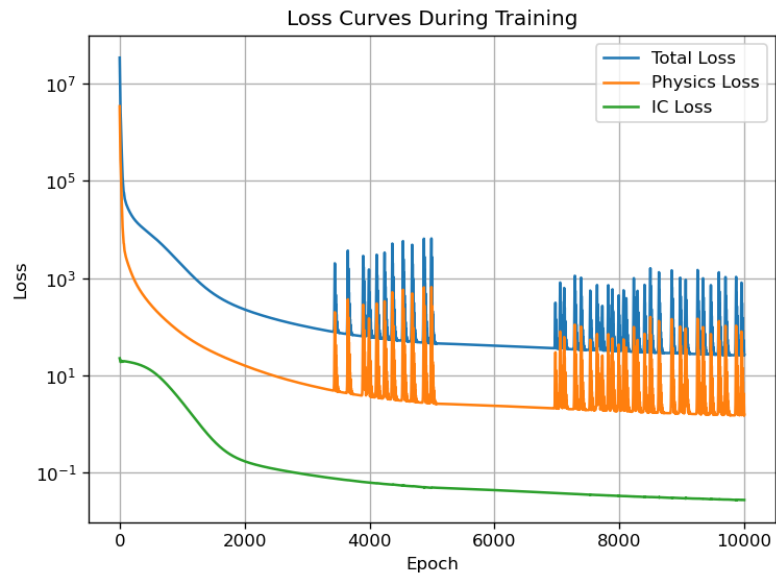


Figure 5.16 The evolution of leaning curve during training with Adam optimizer for baseline models trained on scalar field dynamics with scalar field mass $m_a = 40$, using the same hyperparameter as $m_a = 30$.

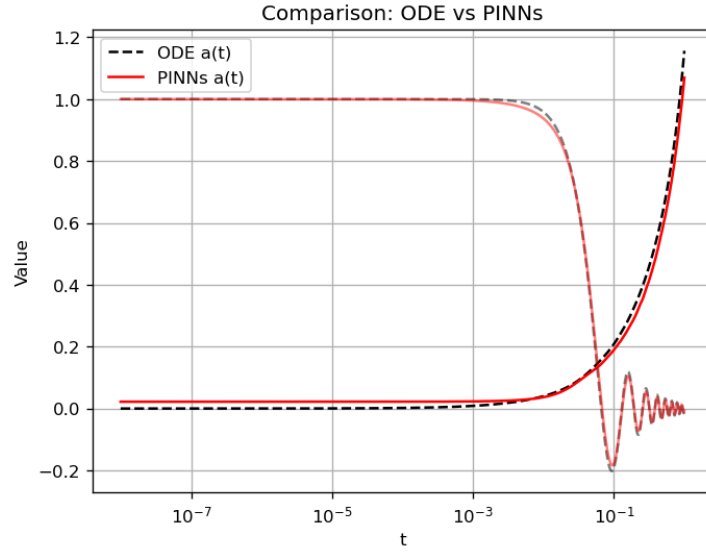


Figure 5.17 Comparison between ODEs and PINNs for $\phi(t)$ (oscillation line) and $a(t)$ (exponential line) with scalar field mass $m_a = 50$, using the same hyperparameter as $m_a = 30$.

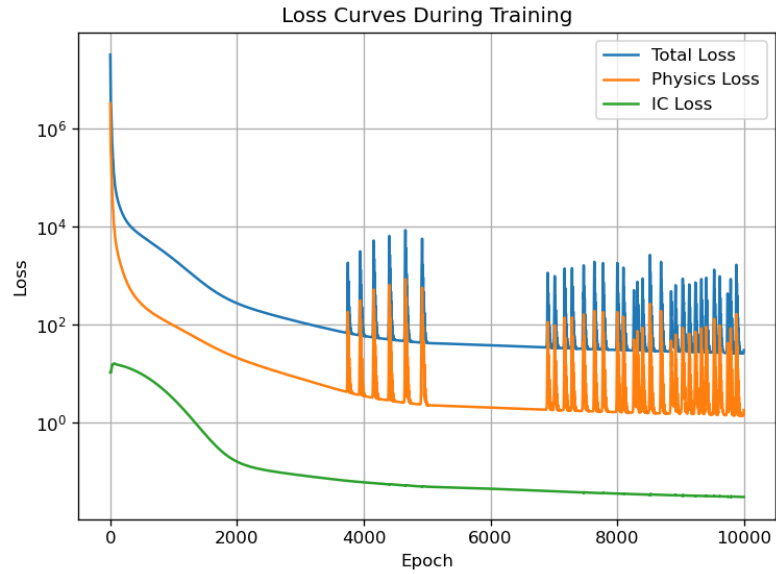


Figure 5.18 The evolution of leaning curve during training with Adam optimizer for baseline models trained on scalar field dynamics with scalar field mass $m_a = 50$, using the same hyperparameter as $m_a = 30$.

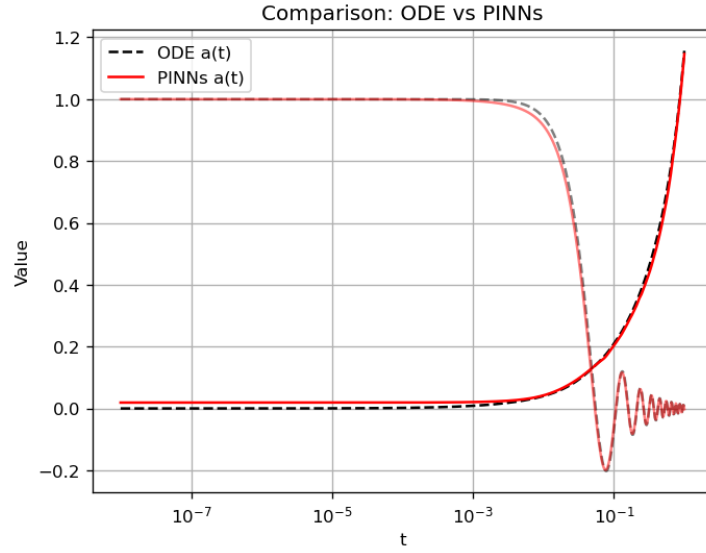


Figure 5.19 Comparison between ODEs and PINNs for $\phi(t)$ (oscillation line) and $a(t)$ (exponential line) with scalar field mass $m_a = 60$, using the same hyperparameter as $m_a = 30$.

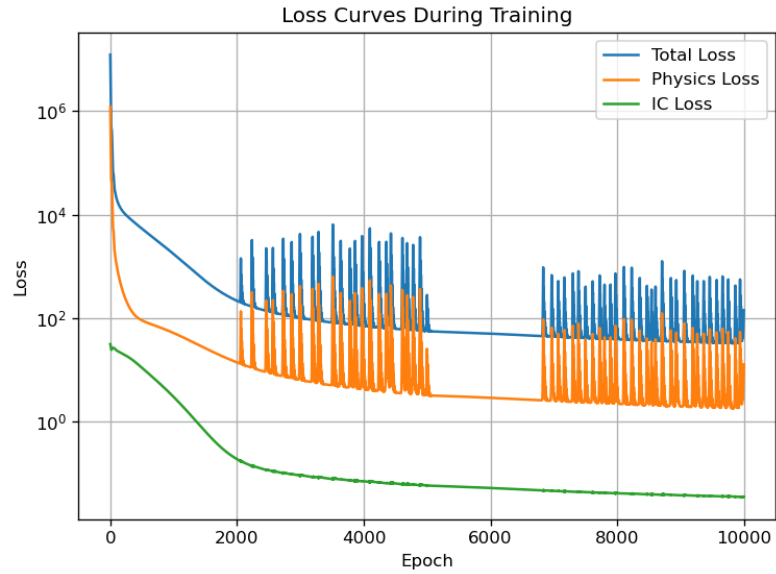


Figure 5.20 The evolution of leaning curve during training with Adam optimizer for baseline models trained on scalar field dynamics with scalar field mass $m_a = 60$, using the same hyperparameter as $m_a = 30$.

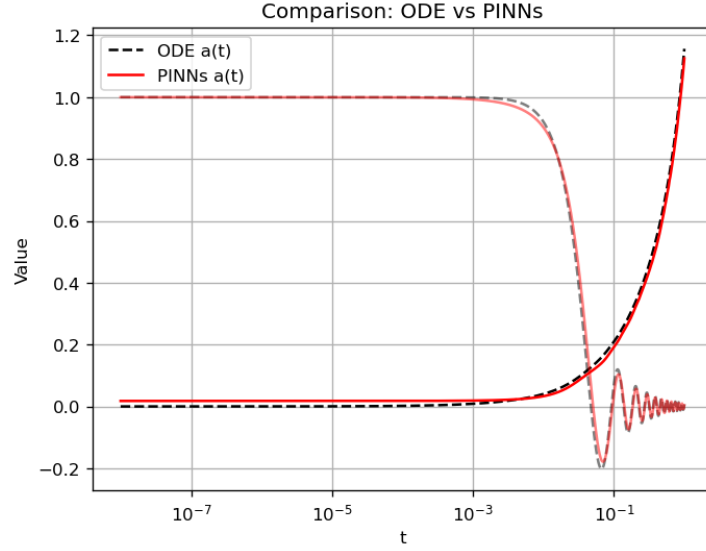


Figure 5.21 Comparison between ODEs and PINNs for $\phi(t)$ (oscillation line) and $a(t)$ (exponential line) with scalar field mass $m_a = 70$.

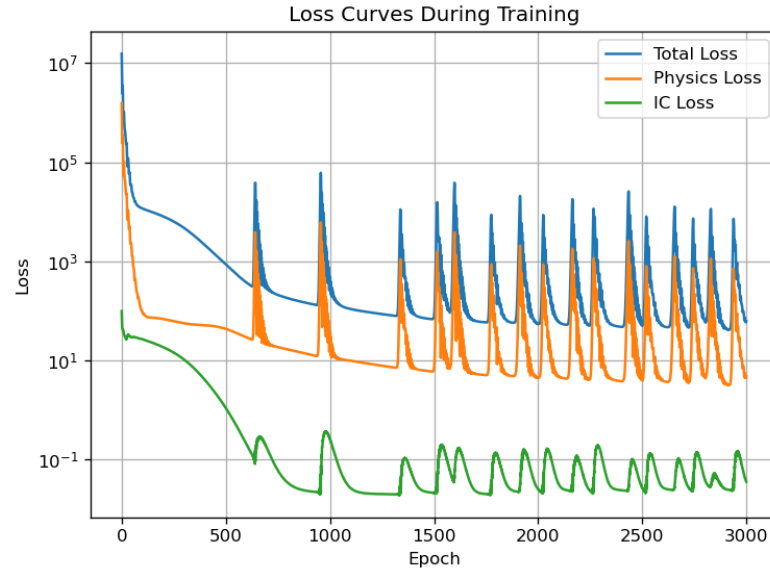


Figure 5.22 The evolution of leaning curve during training with Adam optimizer for baseline models trained on scalar field dynamics with scalar field mass $m_a = 70$.

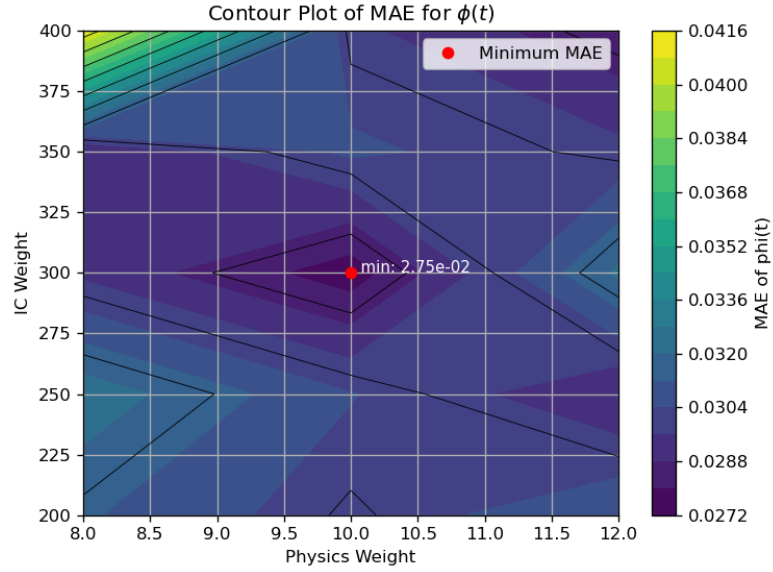


Figure 5.23 Contour plots of the mean absolute error (MAE) for $\phi(t)$ with scalar field mass $m_a = 70$. This is the hyperparameter value obtained from scanning the MAE values for λ_{IC} and λ_{phys} , which yielded the minimum values of the hyperparameters are approximately 2.75×10^{-2} occurs near $\lambda_{IC} = 300$, $\lambda_{phys} = 10$. We re-evaluated the hyperparameter values to find the best MAE fro $m_a = 70$.

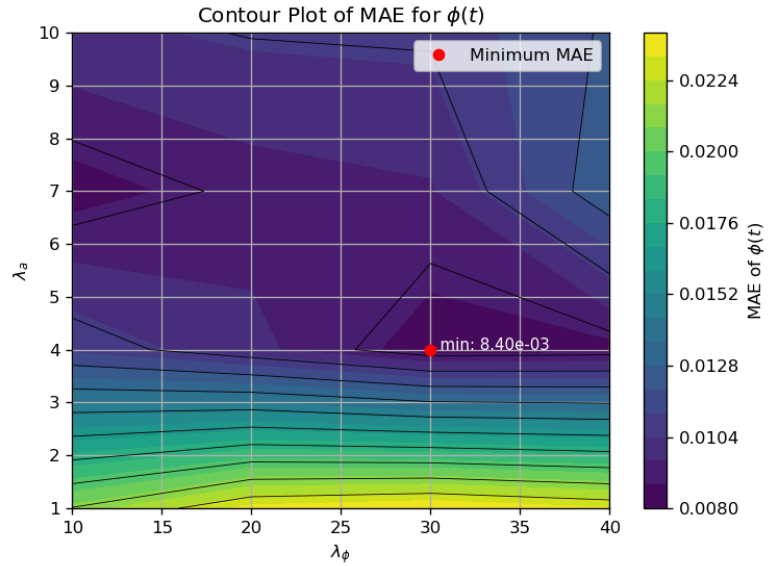


Figure 5.24 Contour plots of the mean absolute error (MAE) for $\phi(t)$ with scalar field mass $m_a = 70$. The MAE of λ_a and λ_ϕ at the minimum point approximately 8.40×10^{-3} at $\lambda_a = 4$, $\lambda_\phi = 30$.

CHAPTER VI

SUMMARY AND OUTLOOK

In this work, we study the Physics-Informed Neural Networks (PINNs) used to solve axion cosmology in the single-field case, where the axion field is described as a scalar degree of freedom evolving in the expanding universe. The dynamics are governed by the coupled Klein–Gordon and Friedmann equations, which encapsulate both the self-consistent evolution of the scalar field and the cosmic scale factor.

To investigate the effectiveness of PINNs in this setting, we considered two different neural network architectures. The first approach of a standard PINNs, where the governing equations are embedded directly into the loss function alongside initial conditions and boundary conditions. This framework allows us to assess the baseline performance of PINNs in capturing the nonlinear dynamics of the axion field. The second approach introduces an enhanced PINNs architecture by incorporating Fourier feature embeddings and residual blocks. Fourier features are employed to overcome the spectral bias of standard neural networks, enabling better representation of oscillatory behavior inherent in axion field dynamics. Residual blocks further stabilize and accelerate training by improving gradient flow in deeper networks. Together, these modifications aim to improve accuracy and robustness, especially during the stiff and highly nonlinear phases of the field evolution.

In order to validate the accuracy of the PINNs based approaches, we compared their results against those obtained from conventional ODE solvers, specifically `scipy.solve_ivp` with the Runge–Kutta method (RK45). The ODE solver serves as a numerical benchmark, providing highly reliable solutions under standard step-size control. Our results show that while RK45 efficiently handles the equations in most regimes, it may become computationally expensive in the presence of stiffness or when very small time steps are required. By contrast, PINNs—though requiring longer training times—offer the advantage of producing global, differentiable approximations to the solution that remain valid across the entire temporal domain. This property makes PINNs especially attractive for problems where analytic

continuity, sparse data, or physical constraints play a critical role.

Overall, this chapter highlights the potential of PINNs as a versatile computational framework for modeling axion cosmology. By comparing the baseline and modified architectures with traditional ODE solvers, we demonstrate both the strengths and current limitations of PINNs in addressing the challenges of nonlinear scalar field evolution. These results provide a foundation for future extensions toward multi-field axion scenarios and more complex models of the early universe.

REFERENCES

1. Marsh DJE. Axion cosmology. **Physics Reports**. 2016 Jul; 643: 1–79.
2. Peccei RD, Quinn HR. CP Conservation in the Presence of Pseudoparticles. **Phys Rev Lett**. 1977 Jun; 38: 1440-3.
3. Weinberg S. A New Light Boson? **Phys Rev Lett**. 1978 Jan; 40: 223-6.
4. Preskill J, Wise MB, Wilczek F. Cosmology of the Invisible Axion. **Phys Lett B**. 1983; 120: 127-32.
5. Stott MJ, Marsh DJE, Pongkitivanichkul C, Price LC, Acharya BS. Spectrum of the axion dark sector. **Phys Rev D**. 2017; 96(8): 083510.
6. Wilczek F. Problem of Strong P and T Invariance in the Presence of Instantons. **Phys Rev Lett**. 1978; 40: 279-82.
7. Abbott LF, Sikivie P. A Cosmological Bound on the Invisible Axion. **Phys Lett B**. 1983; 120: 133-6.
8. Dine M, Fischler W. The Not So Harmless Axion. **Phys Lett B**. 1983; 120: 137-41.
9. Sikivie P. In: Axion Cosmology. **Springer Berlin Heidelberg**; 2008. p. 19–50.
10. Ringwald A. Exploring the Role of Axions and Other WISPs in the Dark Universe. **Phys Dark Univ**. 2012; 1: 116-35.
11. Wantz O, Shellard EPS. Axion cosmology revisited. **Physical Review D**. 2010 Dec; 82(12).
12. Arias P, Cadamuro D, Goodsell M, Jaeckel J, Redondo J, Ringwald A. WISPy cold dark matter. **Journal of Cosmology and Astroparticle Physics**. 2012 Jun; 2012(06): 013–013.
13. Lagaris IE, Likas A, Fotiadis DI. Artificial neural networks for solving ordinary and partial differential equations. **IEEE Transactions on Neural Networks**. 1998; 9(5): 987-1000.

14. Raissi M, Perdikaris P, Karniadakis GE. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. **Journal of Computational Physics**. 2019; 378: 686-707.
15. Chen Z, Liu Y, Sun H. Physics-informed learning of governing equations from scarce data. **Nature Communications**. 2021 Oct; 12(1).
16. Cuomo S, Di Cola VS, Giampaolo F, Rozza G, Raissi M, Piccialli F. Scientific machine learning through physics-informed neural networks: Where we are and what's next. **Journal of Scientific Computing**. 2022; 92(3): 88.
17. Karniadakis G, Kevrekidis I, Lu L, Perdikaris P, Wang S, Yang L. Physics-informed machine learning. **Nature Reviews Physics**. 2021 Jun; 3(6): 422-40. Publisher Copyright: © 2021, Springer Nature Limited.
18. Lu L, Meng X, Mao Z, Karniadakis GE. DeepXDE: A Deep Learning Library for Solving Differential Equations. **SIAM Review**. 2021 Jan; 63(1): 208–228.
19. Luna R, Calderón Bustillo J, Seoane Martínez JJ, Torres-Forné A, Font JA. Solving the Teukolsky equation with physics-informed neural networks. **Physical Review D**. 2023 Mar; 107(6).
20. Luna R, Doneva DD, Font JA, Lien JH, Yazadjiev SS. Quasinormal modes in modified gravity using physics-informed neural networks. **Physical Review D**. 2024 Jun; 109(12).
21. Baumann D. **Cosmology**. Cambridge University Press; 2022.
22. **Summer Student Lecture Programme 2024 at CERN**; [online] 2024 July 12 [cited 2025 Aug 23]. Available from: <https://indico.cern.ch/event/1347523/contributions/5956860/>.
23. Hlozek R, Grin D, Marsh DJE, Ferreira PG. A search for ultralight axions using precision cosmological data. **Physical Review D**. 2015 May; 91(10).
24. **Neural Network Definition and Components**; [online] 2025 [cited 2025 Aug 23]. Available from: <https://mriquestions.com/what-is-a-neural-network.html>.
25. Rosenblatt F. The perceptron: a probabilistic model for information storage and organization in the brain. **Psychological review**. 1958; 65(6): 386.

26. Minsky M, Papert S. An introduction to computational geometry. **Cambridge tiass, HIT**. 1969; 479(480): 104.
27. Cybenko G. Approximation by superpositions of a sigmoidal function. **Mathematics of Control, Signals and Systems**. 1989; 2: 303-14.
28. Hornik K. Approximation capabilities of multilayer feedforward networks. **Neural Networks**. 1991; 4(2): 251-7.
29. Lagaris IE, Likas A, Fotiadis DI. Artificial neural networks for solving ordinary and partial differential equations. **IEEE Transactions on Neural Networks**. 1998; 9(5): 987-1000.
30. Raissi M, Perdikaris P, Karniadakis GE. Physics Informed Deep Learning (Part I): Data-driven solutions of nonlinear partial differential equations. **arXiv preprint arXiv:171110561**. 2017.
31. Raissi M, Perdikaris P, Karniadakis GE. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. **Journal of Computational Physics**. 2019; 378: 686-707.
32. Haghighat E, Juanes R. Physics-informed neural networks for inverse problems in mechanics. **Computer Methods in Applied Mechanics and Engineering**. 2021; 379: 113741.
33. Molina A, coauthors. PINNs for solving the Schrödinger equation. **Physical Review Research**. 2022; 4(3): 033104.
34. Eyama S, Masada Y. Constraining the Cosmological Constant from Stellar Orbits Around Sgr A* Using Physics-Informed Neural Networks. **arXiv preprint arXiv:250819719**. 2025.
35. Varey J, Ruprecht JD, Tierney M, Sullenberger R. Physics-Informed Neural Networks for Satellite State Estimation. In: **2024 IEEE Aerospace Conference**. IEEE; 2024. p. 1–8.
36. Stefanou P, Urbán JF, Pons JA. Solving the pulsar equation using physics-informed neural networks. **Monthly Notices of the Royal Astronomical Society**. 2023 Sep; 526(1): 1504–1511.

37. Dai Z, Moews B, Vilalta R, Dave R. Physics-informed neural networks in the recreation of hydrodynamic simulations from dark matter. **Mon Not Roy Astron Soc.** 2023; 527(2): 3381-94.
38. Korber D, Bianco M, Tolley E, Kneib JP. PINION: physics-informed neural network for accelerating radiative transfer simulations for cosmic reionization. **Monthly Notices of the Royal Astronomical Society.** 2023 Feb; 521(1): 902–915.
39. Chen X, Jeffery DJ, Zhong M, McClenny L, Braga-Neto U, Wang L. Using Physics Informed Neural Networks for Supernova Radiative Transfer Simulation. **arXiv e-prints.** 2022 Nov: arXiv:2211.05219.
40. Ade PAR, et al. Planck 2015 results. XIII. Cosmological parameters. **Astron Astrophys.** 2016; 594: A13.
41. Fixsen DJ, Cheng ES, Gales JM, Mather JC, Shafer RA, Wright EL. The Cosmic Microwave Background Spectrum from the FullCOBEFIRAS Data Set. **The Astrophysical Journal.** 1996 Dec; 473(2): 576–587.
42. Virtanen P, Gommers R, Oliphant TE, Haberland M, Reddy T, Cournapeau D, et al. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. **Nature Methods.** 2020; 17: 261-72.
43. Dormand JR, Prince PJ. A family of embedded Runge-Kutta formulae. **Journal of computational and applied mathematics.** 1980; 6(1): 19-26.
44. Shampine LF. Some practical runge-kutta formulas. **Mathematics of computation.** 1986; 46(173): 135-50.

APPENDICES

APPENDIX A Programming Environment

In this work, we calculated and trained the model on a personal laptop equipped with an AMD Ryzen 5 5600H (12 core), 32 GB of RAM, and an NVIDIA GeForce RTX 3050 Laptop (4 GB VRAM). The implementation was carried out using **Python** language programming and **PyTorch** deep learning framework. All codes in this project are publicly available at https://github.com/Weerachai-00/Axion_Dynamics_using_PINNs. The computational environment is summarized in below.

Component	Specification
Operating System	Ubuntu 22.04 LTS (Linux x86_64)
Programming Language	Python
Deep Learning Framework	PyTorch
Numerical Libraries	NumPy, SciPy, Pandas, Matplotlib
Machine Learning Tools	scikit-learn
Hardware	AMD Ryzen 5 5600H (12 cores, 24 threads)
Memory	32 GB DDR4 RAM
GPU	NVIDIA GeForce RTX 3050 Laptop GPU (4 GB VRAM)

APPENDIX B Code for PINNs model

This is the code for the PINNs model that we used in this work.

```

1 #####
2 # Physics-Informed Neural Network (PINN) Solver for Axion Cosmology
3 # -----
4 # This implementation solves the scale factor  $a(t)$  and
5 # scalar field  $\phi(t)$  using both numerical ODE (solve_ivp)
6 # and Physics-Informed Neural Networks (PINNs).
7 #####
8
9 # Import all used libraries
10 import numpy as np
11 import pandas as pd
12 import torch
13 import torch.nn as nn
14 import torch.optim as optim
15 from scipy.integrate import solve_ivp
16 import matplotlib.pyplot as plt
17 import os
18 import time
19 from sklearn.metrics import mean_squared_error, mean_absolute_error
20
21 #####
22 # Main class for PINNs to use training
23 # and calculation traditional numerical method.
24 #####
25 class PINNSolver:
26     # Check and receive the values of parameters.
27     def __init__(self, N_fields=1, m_vec=None, rho_m0=0.81,
28                 rho_r0=0.00027138, rho_l0=2.19, a0=1e-8,
29                 phi0=None, phi_dot0=None, t_span=(0.0, 1.0),
30                 t_eval=None, device=None, folder_name='results'):
31         """
32         Initializes the solver with default cosmological parameters
33         and model setup.
34         """

```

```

35
36     # Select device automatically (GPU if available)
37     self.device = device or torch.device('cuda' if
38     torch.cuda.is_available() else 'cpu')
39     print(f"Using device: {self.device}")
40
41     # Number of scalar fields (axion fields)
42     self.N_fields = N_fields
43
44     # Mass vector for each axion field (default: all identical)
45     self.m_vec = m_vec if m_vec is not None else
46         np.array([1e2]*N_fields)
47
48     # Energy densities at initial
49     # (matter, radiation, cosmological constant)
50     self.rho_m0 = rho_m0
51     self.rho_r0 = rho_r0
52     self.rho_l0 = rho_l0
53
54     # Initial conditions for scale factor and scalar fields
55     self.a0 = a0
56     self.phi0 = phi0 if phi0 is not None else
57         np.array([1.0]*N_fields)
58     self.phi_dot0 = phi_dot0 if phi_dot0 is not None else
59         np.array([0.0]*N_fields)
60
61     # Combine into single vector for ODE solver: [a,phi,phi_dot]
62     self.y0 = np.concatenate([[self.a0], self.phi0,
63         self.phi_dot0])
64
65     # Time domain setup
66     self.t_span = t_span
67     # Evaluate on log scale to capture earlytime behavior
68     self.t_eval = t_eval if t_eval is not None else
69         np.logspace(np.log10(a0), np.log10(t_span[1]), 1000)
70         .astype(np.float32)
71

```



```

72     # Create output folder
73     self.folder_name = folder_name
74     os.makedirs(self.folder_name, exist_ok=True)
75
76     # Choose PINNs or PINNs+FFT+RB
77     # self.model = PINN(num_layers = 3, num_neurons = 64,
78     #                   N_fields = self.N_fields).to(self.device)
79     self.model = F_PINN(N_fields=self.N_fields, embed_dim=64,
80                         hidden=128, blocks=3, fourier_scale=10.0).to(self.device)
81
82     # Convert physical parameters and ICs to PyTorch tensors
83     self.m_vec_torch = torch.tensor(self.m_vec,
84                                     dtype=torch.float32, device=self.device)
85     self.phi0_torch = torch.tensor(self.phi0.reshape(1, -1),
86                                   device=self.device)
87     self.a0_torch = torch.tensor([[self.a0]],
88                                  device=self.device)
89     self.t0_torch = torch.tensor([[0.0]], device=self.device)
90     print(self.m_vec)
91
92     # General PINNs model
93     class PINN(nn.Module):
94         # Structure of PINNs
95         def __init__(self, num_layers, num_neurons, N_fields):
96             super().__init__()
97             self.layers = nn.ModuleList()
98             self.layers.append(nn.Linear(1, num_neurons))
99             for _ in range(num_layers):
100                 self.layers.append(nn.Linear
101                                   (num_neurons, num_neurons))
102             self.layers.append(nn.Linear(num_neurons, 1+N_fields))
103         # Forward calculation of each layers
104         def forward(self, t):
105             x = t
106             for layer in self.layers[:-1]:
107                 z = layer(x)
108                 x = z * torch.sin(z)

```

```

109         x = self.layers[-1](x)
110         a = torch.nn.functional.softplus(x[:, 0:1])
111         a = torch.clamp(a, min=1e-6)
112         phi = x[:, 1:]
113         return a, phi
114
115 #####
116 # Numerical ODE System (for ground truth)
117 #####
118     def ode_system(self, t, y):
119         """
120         Defines the coupled ODE system for a(t), phi(t),
121         and phidot(t) following the -KleinGordon
122         and Friedmann equations.
123         """
124         N = self.N_fields
125         a = y[0]
126         phi = y[1:N+1]
127         phi_dot = y[N+1:2*N+1]
128
129         # Kinetic and potential energy terms
130         kinetic = 0.5 * np.sum((phi_dot * a)**2)
131         potential = 0.5 * np.sum((self.m_vec**2) * (phi * a)**2)
132
133         # Hubble parameter H(a,phi)
134         H = np.sqrt((1/3) * (kinetic + potential + self.rho_m0 / a
135             + self.rho_r0 / a**2 + self.rho_l0 * a**2))
136
137         # Time derivatives
138         a_dot = H
139         phi_ddot = - np.sqrt(3) * np.sqrt(0.5 * np.sum(phi_dot**2)
140             + 0.5 * np.sum((self.m_vec**2) * phi**2)
141             + self.rho_m0/a**3 + self.rho_r0/a**4 + self.rho_l0)
142             * phi_dot - (self.m_vec**2) * phi
143
144         # Combine derivatives
145         dydt = np.zeros_like(y)

```

```

146     dydt[0] = a_dot
147     dydt[1:N+1] = phi_dot
148     dydt[N+1:2*N+1] = phi_ddot
149     return dydt
150
151 #####
152 # Numerical ODE Solver
153 #####
154     def solve_ode(self):
155         """Solves the ODE system using 'SciPys RK45 solver.'"""
156         print("Solving ODE...")
157         start_ode = time.time()
158         sol = solve_ivp(self.ode_system, self.t_span, self.y0,
159             t_eval=self.t_eval, method='RK45')
160         print(f"ODE solve time: {time.time() - start_ode:.2f} sec")
161
162         # Store numerical solutions for comparison
163         self.a_sol = sol.y[0, :]
164         self.phi_sol = sol.y[1:1+self.N_fields, :]
165
166 #####
167 # Loss function of PINNs
168 #####
169     def physics_loss(self, model, t):
170         """Computes the residual loss from the Friedmann
171         and -KleinGordon equations."""
172         a, phi = model(t)
173
174         # Compute time derivatives using automatic differentiation
175         a_t = torch.autograd.grad(a, t, torch.ones_like(a),
176             create_graph=True)[0]
177         phi_t = torch.autograd.grad(phi, t, torch.ones_like(phi),
178             create_graph=True)[0]
179         phi_tt = torch.autograd.grad(phi_t, t,
180             torch.ones_like(phi_t), create_graph=True)[0]
181
182         # Energy densities (kinetic and potential)

```

```

183     kinetic = 0.5 * torch.sum((phi_t**2)
184         * (a**2), dim=1, keepdim=True)
185     potential = 0.5 * torch.sum((self.m_vec_torch**2)
186         * (phi**2) * (a**2), dim=1, keepdim=True)
187
188     # Friedmann
189     Friedmann = a_t - torch.sqrt((1/3)*(kinetic + potential
190         + self.rho_m0/a + self.rho_r0/a**2 + self.rho_l0*a**2))
191
192     # -KleinGordon
193     sqsumrho = torch.sqrt(torch.full_like(t, 3.0))
194         * torch.sqrt(0.5 * torch.sum(phi_t**2, dim=1,
195         keepdim=True) + 0.5 * torch.sum((self.m_vec_torch**2)
196         * phi**2, dim=1, keepdim=True) + self.rho_m0/a**3
197         + self.rho_r0/a**4 + self.rho_l0)
198     KG = phi_tt + sqsumrho * phi_t + phi * (self.m_vec_torch**2)
199     return torch.mean(Friedmann**2) + torch.mean(KG**2)
200
201 #####
202 # Loss function for initial condition
203 #####
204     def initial_loss(self, model):
205         a_pred0, phi_pred0 = model(self.t0_torch)
206         return 10.0 * torch.mean((a_pred0 - self.a0_torch)**2)
207             + 20.0 * torch.mean((phi_pred0 - self.phi0_torch)**2)
208
209 #####
210 # Training using Adam Optimizer
211 #####
212     def train(self, max_epochs_adam=10000, physics_weight=10.0,
213         ic_weight=400.0, N_f=2000, print_every=500):
214         """Trains the PINN using Adam optimizer with
215         scheduled learning rate decay."""
216
217         optimizer = optim.Adam(self.model.parameters(), lr=1e-4)
218         scheduler = optim.lr_scheduler.StepLR(optimizer,
219         step_size=5000, gamma=0.4)

```

```

220
221     # Store loss history for analysis
222     self.loss_history, self.physics_loss_history,
223         self.ic_loss_history = [], [], []
224     print("Training PINN...")
225     start = time.time()
226
227     for epoch in range(max_epochs_adam):
228         # Generate collocation points uniformly in time
229         t_f = torch.linspace(0.0, 1.0, N_f,
230             device=self.device).reshape(-1, 1).requires_grad_()
231
232         # Compute physics-informed and initial condition losses
233         optimizer.zero_grad()
234         loss_physics_val = self.physics_loss(self.model, t_f)
235         loss_ic_val = self.initial_loss(self.model)
236         loss = physics_weight * loss_physics_val
237             + ic_weight * loss_ic_val
238
239         # Backpropagation
240         loss.backward()
241         optimizer.step()
242         scheduler.step()
243
244         # Record losses
245         self.loss_history.append(loss.item())
246         self.physics_loss_history.append(loss_physics_val.item())
247         self.ic_loss_history.append(loss_ic_val.item())
248
249         if epoch % print_every==0 or epoch==max_epochs_adam-1:
250             percent = 100 * epoch / max_epochs_adam
251             print(f"[Adam] Epoch {epoch:5d}/{max_epochs_adam}
252                 ({percent:5.1f}%) | "
253                 f"Total Loss: {loss.item():.3e} |
254                 Physics: {loss_physics_val.item():.3e} |
255                 IC: {loss_ic_val.item():.3e}")
256

```

```

257         print(f"Training time: {time.time() - start:.2f} sec")
258
259 #####
260 # L-BFGS Fine-Tuning Optimization
261 #####
262     def optimize_lbfgs(self):
263         """Performs second-stage fine-tuning
264         using L-BFGS optimizer."""
265         print("Starting LBFGS optimization...")
266         optimizer_lbfgs = optim.LBFGS(self.model.parameters(),
267                                       lr=1.0, max_iter=5000, tolerance_grad=1e-9,
268                                       tolerance_change=1e-10, history_size=100,
269                                       line_search_fn='strong_wolfe')
270
271         def closure():
272             optimizer_lbfgs.zero_grad()
273             t_f = torch.linspace(0.0, 1.0, 2000,
274                                  device=self.device).reshape(-1, 1).requires_grad_()
275             loss = 10.0 * self.physics_loss(self.model, t_f)
276                   + 400.0 * self.initial_loss(self.model)
277             loss.backward()
278             return loss
279
280         optimizer_lbfgs.step(closure)
281
282 #####
283 # Evaluation and Visualization
284 #####
285     def evaluate(self):
286         """Generates PINN predictions for a(t)
287         and phi(t) across evaluation grid."""
288         t_plot = torch.tensor(self.t_eval,
289                               device=self.device).reshape(-1, 1).requires_grad_(True)
290         a_pred, phi_pred = self.model(t_plot)
291         self.a_pred = a_pred.detach().cpu().numpy().flatten()
292         self.phi_pred = phi_pred.detach().cpu().numpy().T
293

```

```

294 def evaluate_error_metrics(self):
295     """Computes quantitative error metrics (MSE and MAE)
296     vs numerical ODE."""
297     mse_a = mean_squared_error(self.a_sol, self.a_pred)
298     mae_a = mean_absolute_error(self.a_sol, self.a_pred)
299     mse_phi, mae_phi = [], []
300     for i in range(self.N_fields):
301         mse_phi.append(mean_squared_error(self.phi_sol[i],
302                                         self.phi_pred[i]))
303         mae_phi.append(mean_absolute_error(self.phi_sol[i],
304                                         self.phi_pred[i]))
305
306     print("\nEvaluation Metrics:")
307     print(f"  a(t)          -> MSE: {mse_a:.4e},
308           MAE: {mae_a:.4e}")
309     for i, (mse_p, mae_p) in enumerate(zip(mse_phi, mae_phi)):
310         print(f"  phi[{i}](t) -> MSE: {mse_p:.4e},
311              MAE: {mae_p:.4e}")
312
313 def plot_results(self):
314     """Plots ODE and PINN predictions (log scale)."""
315     plt.figure(dpi=120)
316     plt.plot(self.t_eval, self.a_sol, 'k--', label='ODE a(t)')
317     plt.plot(self.t_eval, self.a_pred, 'r-', label='PINNs a(t)')
318     for i in range(min(self.N_fields, 5)):
319         plt.plot(self.t_eval, self.phi_sol[i, :], 'k--', alpha=0.5)
320         plt.plot(self.t_eval, self.phi_pred[i, :], 'r-', alpha=0.5)
321     plt.xscale('log')
322     plt.xlabel('t')
323     plt.ylabel('Value')
324     plt.title('Comparison: ODE vs PINNs')
325     plt.title('m = 100')
326     plt.legend()
327     plt.grid()
328     plt.savefig(os.path.join(self.folder_name, 'comparison.png'))
329     plt.show()
330

```

```

331     def plot_results_linear(self):
332         """Plots ODE and PINN results in linear time scale."""
333         plt.figure(dpi=120)
334         plt.plot(self.t_eval, self.a_sol, 'k--', label='ODE a(t)')
335         plt.plot(self.t_eval, self.a_pred, 'r-', label='PINNs a(t)')
336         for i in range(min(self.N_fields, 5)):
337             plt.plot(self.t_eval, self.phi_sol[i, :], 'k--', alpha=0.5)
338             plt.plot(self.t_eval, self.phi_pred[i, :], 'r-', alpha=0.5)
339         plt.xscale('log')
340         plt.xlabel('t')
341         plt.ylabel('Value')
342         plt.title('Comparison: ODE vs PINNs')
343         plt.legend()
344         plt.grid()
345         plt.savefig(os.path.join(self.folder_name, 'comparison.png'))
346         plt.show()
347
348     def plot_losses(self):
349         """Plots training loss curves."""
350
351         plt.figure(dpi=120)
352         plt.plot(self.loss_history, label='Total Loss')
353         plt.plot(self.physics_loss_history, label='Physics Loss')
354         plt.plot(self.ic_loss_history, label='IC Loss')
355         plt.yscale('log')
356         plt.xlabel('Epoch')
357         plt.ylabel('Loss')
358         plt.title('Loss Curves During Training')
359         plt.legend()
360         plt.grid(True)
361         plt.tight_layout()
362         plt.savefig(os.path.join(self.folder_name, 'loss_curves.png'))
363         plt.show()
364
365     #####
366     # Supporting Network Components
367     #####

```



```

368 class FourierFeatures(nn.Module):
369     """Applies random Fourier feature embedding for
370     better representation of periodic signals."""
371     def __init__(self, in_features=1, out_features=64, scale=10.0):
372         super().__init__()
373         B = torch.randn((out_features, in_features)) * scale
374         self.register_buffer('B', B)
375
376     def forward(self, x):
377         x_proj = 2 * np.pi * x @ self.B.T
378         return torch.cat([torch.sin(x_proj), torch.cos(x_proj)]
379                           ,dim=-1)
380
381 class ResidualBlock(nn.Module):
382     """Residual block with sine activation
383     for stability in deep PINNs."""
384     def __init__(self, width):
385         super().__init__()
386         self.fc1 = nn.Linear(width, width)
387         self.fc2 = nn.Linear(width, width)
388
389     def forward(self, x):
390         identity = x
391         x = torch.sin(self.fc1(x))
392         x = self.fc2(x)
393         return x + identity
394
395 class F_PINN(nn.Module):
396     """Main Fourier-feature-based PINN architecture."""
397     def __init__(self, N_fields, embed_dim=64, hidden=124,
398                 blocks=3, fourier_scale=10.0):
399
400         super().__init__()
401         self.fourier = FourierFeatures(1, embed_dim,
402                                       scale=fourier_scale)
403         layers = [nn.Linear(2 * embed_dim, hidden)]
404         for _ in range(blocks):

```

```

405         layers.append(ResidualBlock(hidden))
406     self.net = nn.Sequential(*layers)
407     self.output = nn.Linear(hidden, 1 + N_fields)
408
409     def forward(self, t):
410         x = self.fourier(t)
411         x = self.net(x)
412         x = self.output(x)
413         a = torch.nn.functional.softplus(x[:, 0:1])
414         a = torch.clamp(a, min=1e-6)
415         phi = x[:, 1:]
416         return a, phi
417
418 #####
419 # Main Execution
420 #####
421 if __name__ == "__main__":
422     solver = PINNSolver()
423     solver.solve_ode() # Solve numerical ODE for reference
424     solver.train() # Train PINN using Adam
425     solver.optimize_lbfgs() # Fine-tune with L-BFGS
426     solver.evaluate() # Evaluate trained model
427     solver.evaluate_error_metrics()
428     solver.plot_results() # Plot comparison
429     solver.plot_results_linear()
430     solver.plot_losses()

```

APPENDIX C Code for hyperparameter tuning

Hyperparameter tuning is an essential step in optimizing the performance of Physics-Informed Neural Networks (PINNs). In this work, a systematic grid search was conducted to identify the optimal balance between the different loss function components, including the physics-informed loss, the initial condition loss, and the relative weights associated with the scale factor $a(t)$ and scalar field $\phi(t)$.

The following Python script performs a multi-dimensional grid search over selected ranges of the hyperparameters. For each parameter combination, the solver is initialized, trained, and evaluated. The results, including the mean absolute errors (MAE) for both $a(t)$ and $\phi(t)$, are recorded and stored for later analysis.

```

1 #####
2 # Hyperparameter tuning for PINN-based Axion Cosmology Solver
3 #####
4
5 if __name__ == "__main__":
6     # Define search ranges for loss term weights
7     physics_weights = np.linspace(8.0, 12.0, 3)
8     ic_weights = np.linspace(200.0, 400.0, 5)
9     a_weights = np.linspace(1.0, 10.0, 4)
10    phi_weights = np.linspace(10.0, 40.0, 4)
11    results = []
12
13    # Nested grid search loops
14    for physics_weight in physics_weights:
15        for ic_weight in ic_weights:
16            for a_weight in a_weights:
17                for phi_weight in phi_weights:
18                    print(f"\nTraining with physics_weight=
19                        {physics_weight}, ic_weights={ic_weight},
20                        a_weight={a_weight}, phi_weight={phi_weight}")
21
22                    # Initialize solver and solve reference ODE
23                    solver = PINNSolver()
24                    solver.solve_ode()

```

```

25
26     # Start timing the training process
27     start = time.time()
28
29     # Train PINN model with current hyperparameters
30     solver.train(physics_weight=physics_weight,
31                 ic_weight=ic_weight, a_weight=a_weight,
32                 phi_weight=phi_weight)
33
34     # Fine-tune with L-BFGS optimizer
35     solver.optimize_lbfgs(a_weight = a_weight,
36                          phi_weight = phi_weight)
37     print(f"Adam + LBFGS training time:
38           {time.time() - start:.2f} sec")
39
40     # Evaluate model predictions and errors
41     solver.evaluate()
42     solver.evaluate_error_metrics()
43     solver.plot_results()
44     solver.plot_losses()
45
46     # Store results for this configuration
47     results.append({
48         'physics_weight': physics_weight,
49         'ic_weight': ic_weight,
50         'a_weight': a_weight,
51         'phi_weight': phi_weight,
52         'mae_a': solver.mae_a,
53         'mae_phi': np.mean(solver.mae_phi)
54     })
55
56     # Save all results to CSV for post-analysis
57     df = pd.DataFrame(results)
58     df.to_csv('grid_search_results_FFT+Block_5_v2.csv', index=False)
59     print("\nGrid search complete. Results saved to
60           'grid_search_results_FFT+Block_70_v3.csv'.")

```

APPENDIX D Code for contour plot

Making a contour plot for checking the minimum value of MAE.

```

1 #####
2 # Contour plot visualization for grid search hyperparameter tuning
3 #####
4
5 import matplotlib.pyplot as plt
6 import pandas as pd
7 import numpy as np
8
9 # Load grid search results from CSV file
10 df = pd.read_csv('grid_search_results_FFT+Block_70_v3.csv')
11 # Check lowest MAE
12 best_mae_phi_row = df.loc[df['mae_phi'].idxmin()]
13 print("Best (lowest) MAE for phi(t):")
14 print(best_mae_phi_row)
15
16 # Pivot to make a matrix for contouring (a_weight&phi_weight)
17 pivot_phi = df.pivot_table(index="a_weight", columns="phi_weight",
18     values="mae_phi")
19 X, Y = np.meshgrid(pivot_phi.columns.values.astype(float),
20     pivot_phi.index.values.astype(float))
21 Z = pivot_phi.values.astype(float)
22
23 # Plot
24 plt.figure(dpi=120)
25 cp = plt.contourf(X, Y, Z, levels=20, cmap='viridis')
26 plt.colorbar(cp, label=r"MAE of $\phi(t)$")
27 plt.contour(X, Y, Z, levels=10, colors='black', linewidths=0.5)
28
29 # Mark the global minimum
30 min_idx = np.unravel_index(np.argmin(Z), Z.shape)
31 min_ic = X[min_idx]
32 min_phys = Y[min_idx]
33 plt.plot(min_ic, min_phys, 'ro', label='Minimum MAE')
34 plt.text(min_ic, min_phys, f"min: {Z[min_idx]:.2e}", color='white',

```

```

35     fontsize=9)
36
37 plt.xlabel(r"$\lambda_{\phi}$")
38 plt.ylabel(r"$\lambda_a$")
39 plt.title(r"Contour Plot of MAE for $\phi(t)$")
40 plt.legend()
41 plt.grid(True)
42 plt.tight_layout()
43 plt.show()
44
45 # Pivot to make a matrix for contouring (ic_weight&physics_weight)
46 pivot_a = df.pivot_table(index="ic_weight", columns="physics_weight",
47     values="mae_a")
48 X, Y = np.meshgrid(pivot_a.columns.values.astype(float),
49     pivot_a.index.values.astype(float))
50 Z = pivot_a.values.astype(float)
51
52 plt.figure(dpi=120)
53 cp = plt.contourf(X, Y, Z, levels=20, cmap='viridis')
54 plt.colorbar(cp, label="MAE of phi(t)")
55 plt.contour(X, Y, Z, levels=10, colors='black', linewidths=0.5)
56
57 # Mark the global minimum
58 min_idx = np.unravel_index(np.argmin(Z), Z.shape)
59 min_ic = X[min_idx]
60 min_phys = Y[min_idx]
61 plt.plot(min_ic, min_phys, 'ro', label='Minimum MAE')
62 plt.text(min_ic, min_phys, f"min: {Z[min_idx]:.2e}", color='white',
63     fontsize=9)
64
65 plt.ylabel("IC Weight")
66 plt.xlabel("Physics Weight")
67 plt.title(r"Contour Plot of MAE for $\phi(t)$")
68 plt.legend()
69 plt.grid(True)
70 plt.tight_layout()
71 plt.show()

```